



Gowin HDL 编码风格 用户指南

SUG949-2.2, 2026-06-17

版权所有 © 2026 广东高云半导体科技股份有限公司

GOWIN高云、、Gowin、GowinSynthesis、晨熙以及小蜜蜂以及高云均为广东高云半导体科技股份有限公司注册商标，本手册中提到的其他任何商标，其所有权利属其拥有者所有。未经本公司书面许可，任何单位和个人都不得擅自摘抄、复制、翻译本文档内容的部分或全部，并不得以任何形式传播。

免责声明

本文档并未授予任何知识产权的许可，并未以明示或暗示，或以禁止反言或其它方式授予任何知识产权许可。除高云半导体在其产品的销售条款和条件中声明的责任之外，高云半导体概不承担任何法律或非法律责任。高云半导体对高云半导体产品的销售和 / 或使用不作任何明示或暗示的担保，包括对产品的特定用途适用性、适销性或对任何专利权、版权或其它知识产权的侵权责任等，均不作担保。高云半导体对文档中包含的文字、图片及其它内容的准确性和完整性不承担任何法律或非法律责任，高云半导体保留修改文档中任何内容的权利，恕不另行通知。高云半导体不承诺对这些文档进行适时的更新。

版本信息

日期	版本	说明
2020/08/31	1.0	初始版本。
2021/06/10	1.1	添加单端 BUF 的 infer 案例。
2022/05/31	1.2	删除编码风格要求描述。
2025/04/30	1.8	● 新增 VHDL 示例； ● 删除 SP 的 Decoder 形式。
2025/08/29	2.0	● 更新 2 简介章节； ● 更新 3 触发器章节； ● 更新 4 锁存器章节； ● 更新 5 三态输出缓冲器章节； ● 更新 6 双向缓冲器章节； ● 更新 7 移位寄存器章节； ● 更新 8 RAM 章节； ● 更新 9 ROM 章节； ● 更新 10 DSP 章节； ● 更新 11 FSM 章节。
2025/12/31	2.1	更新部分 VHDL 案例的实体名。
2026/06/17	2.2	● 第 3 章触发器、第 4 章锁存器、第 5 章三态输出缓冲器、第 6 章双向缓冲器案例中新增注释； ● 调整第 10 DSP 章乘法器、乘加器、预加操作描述顺序； ● 修改、新增异步清零和锁存使能的数据锁存器案例。

目录

目录	i
表目录	v
1 关于本手册	1
1.1 手册内容	1
1.2 相关文档	1
1.3 术语、缩略语	2
1.4 技术支持与反馈	2
2 简介	3
3 触发器	4
3.1 GowinSynthesis 可识别的触发器	4
3.2 编码指南	4
3.3 触发器示例	4
3.3.1 具有时钟使能和同步置位的触发器（Verilog）	4
3.3.2 具有时钟使能和同步置位的触发器（VHDL）	5
3.3.3 具有时钟使能和同步复位的触发器（Verilog）	6
3.3.4 具有时钟使能和同步复位的触发器（VHDL）	6
3.3.5 具有时钟使能和异步置位的触发器（Verilog）	7
3.3.6 具有时钟使能和异步置位的触发器（VHDL）	8
3.3.7 具有时钟使能和异步清零的触发器（Verilog）	8
3.3.8 具有时钟使能和异步清零的触发器（VHDL）	9
4 锁存器	11
4.1 锁存器	11
4.2 锁存器示例	11
4.2.1 具有异步清零和锁存使能的数据锁存器（Verilog）	11
4.2.2 具有异步清零和锁存使能的数据锁存器（VHDL）	12
4.2.3 具有异步置位和锁存使能的数据锁存器（Verilog）	12
4.2.4 具有异步置位和锁存使能的数据锁存器（VHDL）	13

5 三态输出缓冲器	15
5.1 三态输出缓冲器	15
5.2 三态输出缓冲器示例	15
5.2.1 连续赋值形式的三态输出缓冲器 (Verilog)	15
5.2.2 连续赋值形式的三态输出缓冲器 (VHDL)	15
5.2.3 组合逻辑形式的三态输出缓冲器 (Verilog)	16
5.2.4 组合逻辑形式的三态输出缓冲器 (VHDL)	16
6 双向缓冲器	18
6.1 双向缓冲器	18
6.2 双向缓冲器示例	18
6.2.1 连续赋值形式的双向缓冲器 (Verilog)	18
6.2.2 连续赋值形式的双向缓冲器 (VHDL)	18
6.2.3 组合逻辑形式的双向缓冲器 (Verilog)	19
6.2.4 组合逻辑形式的双向缓冲器 (VHDL)	19
7 移位寄存器	21
7.1 静态移位寄存器	21
7.1.1 静态移位寄存器	21
7.1.2 静态移位寄存器示例	21
7.2 动态移位寄存器	24
7.2.1 动态移位寄存器	24
7.2.2 动态移位寄存器示例	25
8 RAM	29
8.1 块状静态随机存储器 (BSRAM)	29
8.1.1 单端口模式的 BSRAM 示例	29
8.1.2 伪双端口模式的 BSRAM 示例	38
8.1.3 双端口模式的 BSRAM 示例	50
8.1.4 Byte-enable 功能	60
8.2 分布式静态随机存储器 (SSRAM)	75
8.2.1 具有同步读取功能的单端口模式 SSRAM 示例	75
8.2.2 具有同步读取功能的伪双端口模式的 SSRAM 示例	77
8.3 RAM 初始化	79
8.3.1 HDL 源代码中指定 RAM 初始化内容	79
8.3.2 外部数据文件中指定 RAM 初始化内容	82
9 ROM	89
9.1 ROM 示例	89

9.1.1 定义时赋初值的 ROM (Verilog)	89
9.1.2 定义时赋初值的 ROM (VHDL)	90
9.1.3 Case 形式的 ROM (Verilog)	92
9.1.4 Case 形式的 ROM (VHDL)	95
9.1.5 initial 块赋初值的 ROM (Verilog)	98
9.1.6 readmemb 赋初值的 ROM (Verilog)	99
9.1.7 readmemb 赋初值的 ROM (Verilog)	101
9.1.8 readln 赋初值的 ROM (VHDL)	104
10 DSP	108
10.1 乘法器	108
10.1.1 乘法器示例	108
10.2 乘加器	114
10.2.1 静态乘加/减示例	114
10.2.2 动态乘加/减示例	121
10.3 累加器	129
10.3.1 静态累加示例	129
10.3.2 动态累加示例	137
10.4 预加操作	144
10.4.1 静态预加/减操作示例	144
10.4.2 动态预加/减操作示例	150
10.5 级联操作	157
10.5.1 静态级联示例	157
10.5.2 动态级联示例	166
10.6 移位操作	174
10.6.1 移位操作示例	174
10.7 算术逻辑操作	178
10.7.1 算数逻辑示例	178
11 FSM	182
11.1 FSM 描述	182
11.2 编码方式	182
11.3 FSM 示例	182
11.3.1 独热码编码方式的 Moore 型状态机 (Verilog)	182
11.3.2 独热码编码方式的 Moore 型状态机 (VHDL)	185
11.3.3 独热码编码方式的 Mealy 型状态机 (Verilog)	187
11.3.4 独热码编码方式的 Mealy 型状态机 (VHDL)	190

11.3.5 格雷码编码方式的 Moore 型状态机 (Verilog)	192
11.3.6 格雷码编码方式的 Moore 型状态机 (VHDL)	195
11.3.7 格雷码编码方式的 Mealy 型状态机 (Verilog)	197
11.3.8 格雷码编码方式的 Mealy 型状态机 (VHDL)	200
11.3.9 顺序码编码方式的 Moore 型状态机 (Verilog)	202
11.3.10 顺序码编码方式的 Moore 型状态机 (VHDL)	205
11.3.11 顺序码编码方式的 Mealy 型状态机 (Verilog)	207
11.3.12 顺序码编码方式的 Mealy 型状态机 (VHDL)	210

表目录

表 1-1 术语、缩略语 2

1 关于本手册

1.1 手册内容

本手册主要描述高云 HDL 编码风格要求及原语的 HDL 编码实现，旨在帮助用户快速熟悉高云 HDL 编码风格和原语实现，指导用户设计，提高设计效率。

1.2 相关文档

通过登录高云半导体网站 www.gowinsemi.com.cn 可下载、查看以下相关文档：

- [SUG100, Gowin 云源软件用户指南](#)
- [SUG283, Gowin 原语用户指南](#)
- [UG285, Gowin 存储器\(BSRAM & SSRAM\)用户指南](#)
- [UG287, Gowin 数字信号处理\(DSP\)模块用户指南](#)
- [UG288, Gowin 可配置功能单元\(CFU\)用户指南](#)
- [UG289, Gowin 可编程通用管脚\(GPIO\)用户指南](#)
- [UG300, Arora V 存储器\(BSRAM & SSRAM\)用户指南](#)
- [UG303, Arora V 可配置功能单元\(CFU\)用户指南](#)
- [UG304, Arora V 可编程通用管脚\(GPIO\)用户指南](#)
- [UG305, Arora V 数字信号处理\(DSP\)模块用户指南](#)

1.3 术语、缩略语

表 1-1 中列出了本手册中出现的相关术语、缩略语及相关释义。

表 1-1 术语、缩略语

术语、缩略语	全称	含义
BSRAM	Block Static Random Access Memory	块状静态随机存储器
CLU	Configurable Logic Unit	可配置逻辑单元
DSP	Digital Signal Processing	数字信号处理
FSM	Finite State Machine	有限状态机
HDL	Hardware Description Language	硬件描述语言
LUT	Look-up Table	查找表
SSRAM	Shadow Static Random Access Memory	分布式静态随机存储器

1.4 技术支持与反馈

高云半导体提供全方位技术支持，在使用过程中如有任何疑问或建议，可直接与公司联系：

网址：www.gowinsemi.com.cn

E-mail：support@gowinsemi.com

Tel: +86 755 8262 0391

2简介

HDL 编码技术是数字逻辑设计的核心手段，通过代码精确描述电路行为与结构，并高效映射到目标器件硬件资源。本指南将帮助用户：

- 实现常见数字逻辑电路功能

描述组合逻辑（如查找表、多路复用器）、时序逻辑（如寄存器）及复杂状态机。并提供可综合的 RTL 模板，避免仿真与硬件实现不一致。

- 适配 Gowin 器件架构

利用器件专用资源（如 Block RAM、DSP 单元）的优化编码方法。

通过规范化的 HDL 设计，开发者可充分发挥 Gowin FPGA 的性能与能效优势，降低后期调试风险。高云 HDL 编码总体要求，参考文档 [SUG113, Gowin 器件设计优化与分析用户指南](#) 的第 2 章节。

本文包含编码示例，并支持从中下载对应的示例文件。

3 触发器

3.1 GowinSynthesis 可识别的触发器

GowinSynthesis 可识别具有以下控制信号的触发器：

- 上升沿或下降沿时钟
- 异步置位/复位
- 同步置位/复位
- 时钟使能

3.2 编码指南

1. 声明寄存器时，需给寄存器赋初值。
2. 不要编写同时具有复位和置位功能的寄存器。
3. 若寄存器有初值，则寄存器的复位/置位值需与寄存器的初值相同。
4. 使用非阻塞赋值描述寄存器。
5. 时序逻辑电路的敏感信号不能超过 2 个。

3.3 触发器示例

3.3.1 有时钟使能和同步置位的触发器（Verilog）

```
// 1-bit Register with Rising-edge Clock
// Active-high Synchronous Set
// Active-high Clock Enable
//DFFSE primitive will be inferred by GowinSynthesis.
module gowin_dffse (clk, d, ce, set, q );
input clk, d, ce, set;
output reg q=1'b1;
always @(posedge clk)begin
```

```

        if (set)begin
            q <= 1'b1;
        end
        else begin
            if (ce)begin
                q <= d;
            end
        end
    end
end
endmodule

```

3.3.2 具有时钟使能和同步置位的触发器（VHDL）

```

-- 1-bit Register with Rising-edge Clock
-- Active-high Synchronous Set
-- Active-high Clock Enable
-- DFFSE primitive will be inferred by GowinSynthesis.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity gowin_dffse IS
    PORT(q:OUT std_logic;
         d :IN  std_logic;
         set:IN  std_logic;
         ce:IN  std_logic;
         clk:IN std_logic
    );
end gowin_dffse;
ARCHITECTURE rtl OF gowin_dffse IS
begin
    process(clk)
    begin
        if(clk 'event and clk ='1') then
            if(set='1')then
                q<='1';
            elsif(ce='1')then
                q<=d;
            end if
        end if
    end process
end rtl;

```

```

        end if;
    end if;
end process;
end;

```

3.3.3 具有时钟使能和同步复位的触发器（Verilog）

```

// 1-bit rising-edge clock register
// Active-high synchronous reset
// Active-high clock enable
//DFFRE primitive will be inferred by GowinSynthesis.
module gowin_dffre (clk, d, ce, rst, q );
input clk, d, ce, rst;
output reg q= 1'b0;
always @(posedge clk)begin
    if (rst)begin
        q <= 1'b0;
    end
    else begin
        if (ce)begin
            q <= d;
        end
    end
end
endmodule

```

3.3.4 具有时钟使能和同步复位的触发器（VHDL）

```

-- 1-bit rising-edge clock register
-- Active-high synchronous reset
-- Active-high clock enable
-- DFFRE primitive will be inferred by GowinSynthesis.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity gowin_dffre IS
    PORT(q:OUT std_logic;
        d :IN  std_logic;
        rst:IN  std_logic;

```

```

        ce: IN  std_logic;
        clk:IN std_logic
    );
end gowin_dffre;
ARCHITECTURE rtl OF gowin_dffre IS
begin
    process(clk,rst,ce)
    begin
        if(clk 'event and clk ='1') then
            if(rst='1')then
                q<='0';
            elsif(ce='1')then
                q<=D;
            end if;
        end if;
    end process;
end;

```

3.3.5 具有时钟使能和异步置位的触发器（Verilog）

```

// 1-bit rising-edge clock register
// Active-high asynchronous preset
// Active-high clock enable
//DFFPE primitive will be inferred by GowinSynthesis.
module gowin_dffpe(clk, d, ce, preset, q );
input clk, d, ce, preset;
output reg q= 1'b1;
always @(posedge clk or posedge preset )begin
    if (preset)begin
        q <= 1'b1;
    end
    else begin
        if (ce)begin
            q <= d;
        end
    end
end

```

```

end
endmodule

```

3.3.6 具有时钟使能和异步置位的触发器（VHDL）

```

-- 1-bit rising-edge clock register
-- Active-high asynchronous preset
-- Active-high clock enable
-- DFFPE primitive will be inferred by GowinSynthesis.
library ieee;
use ieee.std_logic_1164.all;
entity gowin_dffpe IS
PORT(
    q:OUT std_logic;
    d: IN  std_logic;
    clk:IN std_logic;
    ce: IN std_logic;
    preset:IN std_logic
);
end entity;
ARCHITECTURE rtl OF gowin_dffpe IS
begin
    process(clk,preset,ce)
    begin
        if(preset='1')then
            q<='1';
        elsif(clk 'event and clk ='1') then
            if(ce='1') then
                q<=D;
            end if;
        end if;
    end process;
end;

```

3.3.7 具有时钟使能和异步清零的触发器（Verilog）

```

// 1-bit rising-edge clock register
// Active-high asynchronous clear

```

```

// Active-high clock enable
//DFFCE primitive will be inferred by GowinSynthesis.
module gowin_dffce (clk, d, ce, clear, q );
input clk, d, ce, clear;
output reg q= 1'b0;
always @(posedge clk or posedge clear )begin
    if (clear)begin
        q <= 1'b0;
    end
    else begin
        if (ce)begin
            q <= d;
        end
    end
end
endmodule

```

3.3.8 具有时钟使能和异步清零的触发器（VHDL）

```

-- 1-bit rising-edge clock register
-- Active-high asynchronous clear
-- Active-high clock enable
--DFFCE primitive will be inferred by GowinSynthesis.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity gowin_dffce IS
    PORT(q:OUT std_logic;
        d : std_logic;
        clk:IN std_logic;
        ce: IN std_logic;
        clear:IN std_logic
    );
end gowin_dffce;
ARCHITECTURE rtl OF gowin_dffce IS
begin
    process(clk,clear,ce)

```

```
begin
    if(clear='1')then
        q<='0';
    elsif(clk 'event and clk ='1') then
        if(ce='1') then
            q<=d;
        end if;
    end if;
end process;
end;
```

4锁存器

4.1 锁存器

锁存器一般是由以下情况导致：

1. 组合逻辑 **case** 语句条件不全情况
2. 组合逻辑 **if-else** 语句不完整
3. 连续赋值或组合逻辑存在环路

4.2 锁存器示例

4.2.1 具有异步清零和锁存使能的数据锁存器（Verilog）

```
// Latch with Positive Gate, Asynchronous Clear, and Latch Enable
// Incomplete case statement conditions lead to latch
// DLCE primitive will be inferred by GowinSynthesis.
module gowin_dlce (q,d,g, ce,clr);
input d,g, ce,clr;
output reg q= 1'b0;
always @(d or g or clr or ce) begin
    if(clr)begin
        q <= 1'b0;
    end
    else begin
        if (g && ce)begin
            q <= d;
        end
    end
end
end
```

```
endmodule
```

4.2.2 具有异步清零和锁存使能的数据锁存器（VHDL）

```
--Latch with Positive Gate, Asynchronous Clear, and Latch Enable
-- Incomplete case statement conditions lead to latch
-- DLCE primitive will be inferred by GowinSynthesis.
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY gowin_dlce IS
PORT(d:IN STD_LOGIC;
      g:IN STD_LOGIC;
      clr:IN STD_LOGIC;
      q:OUT STD_LOGIC;
      ce:IN STD_LOGIC
);
END gowin_dlce;
ARCHITECTURE BEH OF gowin_dlce IS
BEGIN
    PROCESS(g,d,clr,ce)
    BEGIN
        IF (clr='1')THEN
            q <= '0';
        ELSIF(g='1'AND ce='1')THEN
            q<=d;
        END IF;
    END PROCESS;
END;
```

4.2.3 具有异步置位和锁存使能的数据锁存器（Verilog）

```
// Latch with Positive Gate, Asynchronous Preset, and Latch Enable
// Incomplete combinational logic if-else statements lead to latch
// DLPE primitive will be inferred by GowinSynthesis.
module gowin_dlpe (q,d,g, ce,preset);
input d,g, ce,preset;
output reg q= 1'b1;
```

```
always @(d or g or preset or ce) begin
    if(preset)begin
        q <= 1'b1;
    end
    else begin
        if (g && ce)begin
            q <= d;
        end
    end
end
endmodule
```

4.2.4 具有异步置位和锁存使能的数据锁存器（VHDL）

```
-- Latch with Positive Gate, Asynchronous Preset, and Latch Enable
-- Incomplete combinational logic if-else statements lead to latch
-- DLPE primitive will be inferred by GowinSynthesis.
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY gowin_dlpe IS
    PORT(d:IN STD_LOGIC;
        g:IN STD_LOGIC;
        preset:IN STD_LOGIC;
        q:OUT STD_LOGIC;
        ce:IN STD_LOGIC
    );
END gowin_dlpe;
ARCHITECTURE BEH OF gowin_dlpe IS
    BEGIN
        PROCESS(g,d,preset,ce)
        BEGIN
            IF (preset='1')THEN
                q <= '1';
            ELSIF(g='1'AND ce='1')THEN
                q<=d;
            END IF;
        END PROCESS
    END
```

```
END PROCESS;  
END;
```

5 三态输出缓冲器

5.1 三态输出缓冲器

GowinSynthesis 可以识别以下三态缓冲器的编码方式:

1. 通过连续语句进行建模
2. 通过 if-else 结构建模

5.2 三态输出缓冲器示例

5.2.1 连续赋值形式的三态输出缓冲器 (Verilog)

```
// Tristate Description Using Concurrent Assignment
// TBUF primitive will be inferred by GowinSynthesis.
module gowin_tbuf (i, oen, o);
input i,oen;
output o;
assign o= ~oen ? i :1'bz;
endmodule
```

5.2.2 连续赋值形式的三态输出缓冲器 (VHDL)

```
-- Tristate Description Using Concurrent Assignment
-- TBUF primitive will be inferred by GowinSynthesis.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity gowin_tbuf is
    PORT (
        o  : out std_logic;
        i  : in   std_logic;
        oen : in   std_logic
```

```

    );
end gowin_tbuf;
architecture Behavioral of gowin_tbuf is
begin
    o <= i when oen = '0' else 'Z';
end Behavioral;

```

5.2.3 组合逻辑形式的三态输出缓冲器（Verilog）

```

// Tristate Description Using Combinatorial Always Block
// TBUF primitive will be inferred by GowinSynthesis.
module gowin_tbuf (i, oen, o);
input i,oen;
output reg o;
reg data_out;
always@(oen,i)begin
    if(~oen)
        o <=i;
    else
        o <=1'bz;
end
endmodule

```

5.2.4 组合逻辑形式的三态输出缓冲器（VHDL）

```

-- Tristate Description Using Combinatorial Process
-- TBUF primitive will be inferred by GowinSynthesis.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity gowin_tbuf is
    PORT (
        o : OUT    std_logic;
        i : IN     std_logic;
        oen : IN    std_logic
    );
end gowin_tbuf;
architecture Behavioral of gowin_tbuf is
begin

```

```
    process( i, oen) begin
    if ((oen = '0') or (oen = 'L')) then
        o <= i;
    elsif ((oen = '1') or (oen = 'H')) then
        o <= 'Z';
    else
        o <= 'X';
    end if;
    end process;
end Behavioral;
```

6 双向缓冲器

6.1 双向缓冲器

GowinSynthesis 可以识别以下双向缓冲器的编码方式:

1. 通过连续语句进行建模
2. 通过 if-else 结构建模

6.2 双向缓冲器示例

6.2.1 连续赋值形式的双向缓冲器（Verilog）

```
// Bidirectional Description Using Concurrent Assignment
// IOBUF primitive will be inferred by GowinSynthesis.
module gowin_iobuf (i, oen, io, o);
input i,oen;
output o;
inout io;
assign io= ~oen ? i :1'bz;
assign o = io;
endmodule
```

6.2.2 连续赋值形式的双向缓冲器（VHDL）

```
-- Bidirectional Description Using Concurrent Assignment
-- IOBUF primitive will be inferred by GowinSynthesis.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity gowin_iobuf is
    PORT (
```

```

        o : OUT    std_logic;
        io : INOUT std_logic;
        i  : IN     std_logic;
        oen : IN    std_logic
    );
end entity;
architecture Behavioral of gowin_iobuf is
begin
    o<=io;
    io <= i when oen = '0' else  'Z';
end Behavioral;

```

6.2.3 组合逻辑形式的双向缓冲器（Verilog）

```

// Bidirectional Description Using Combinatorial Always Block
// IOBUF primitive will be inferred by GowinSynthesis.
module gowin_iobuf (i, oen,  io, o);
input i,oen;
output  o;
inout io;
reg data_out;
always@(oen,i)begin
    if(~oen)
        data_out<=i;
    else
        data_out<=1'bz;
end
assign io= data_out;
assign o = io;
endmodule

```

6.2.4 组合逻辑形式的双向缓冲器（VHDL）

```

-- Bidirectional Description Using Combinatorial Process
-- IOBUF primitive will be inferred by GowinSynthesis.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```
entity gowin_iobuf is
  PORT (
    o : OUT    std_logic;
    io : INOUT std_logic;
    i  : IN     std_logic;
    oen : IN    std_logic
  );
end entity;

architecture Behavioral of gowin_iobuf is
begin
  process(io, i, oen)
  begin
    o <= TO_X01(io);
    if ((oen = '0') or (oen = 'L')) then
      io <= i;
    elsif ((oen = '1') or (oen = 'H')) then
      io <= 'Z';
    else
      io <= 'X';
    end if;
  end process;

end Behavioral;
```

7 移位寄存器

7.1 静态移位寄存器

静态移位寄存器是由一系列触发器组成的，允许数据在固定数量的延迟级之间传输。

7.1.1 静态移位寄存器

静态移位寄存器通常包含以下元素：

- 时钟
- 可选时钟使能
- 串行数据输入
- 串行数据输出

7.1.2 静态移位寄存器示例

位拼接形式的 32 位移位寄存器（Verilog）

```
// 32-bit Shift Register
// Rising edge clock
// Active high clock enable
module gowin_shfitreg (clk,ce,din,dout);
parameter depth=32;
input clk;
input ce;
input din;
output dout;

reg [depth-1:0]shreg;

always@(posedge clk)begin
```

```

        if(ce)begin
            shreg<={shreg[depth-2:0],din};
        end
    end
    assign dout=shreg[depth-1];
endmodule

```

位拼接形式的 32 位移位寄存器（VHDL）

```

-- 32-bit Shift Register
-- Rising edge clock
-- Active high clock enable
library ieee;
use ieee.std_logic_1164.all;
entity gowin_shfitreg is
    generic(
        DEPTH : integer := 32
    );
    port(
        clk : in std_logic;
        din : in std_logic;
        ce : in std_logic;
        dout : out std_logic
    );
end entity;

architecture rtl of gowin_shfitreg is
    signal shreg : std_logic_vector(DEPTH - 1 downto 0);
begin
    process(clk)
    begin
        if rising_edge(clk) then
            if ce = '1' then
                shreg <= shreg(DEPTH - 2 downto 0) & din;
            end if;
        end if;
    end process;
end architecture;

```

```

        dout <= shreg(DEPTH - 1);
    end rtl;

```

循环形式的 32 位移位寄存器 (Verilog)

```

// 32-bit shift register
// Rising-edge clock
// Active-high clock enable
module gowin_shfitreg (clk, ce, din, dout);
    parameter depth=32;
    input clk, ce;
    input  din;
    output dout;
    reg [depth-1:0]regBank;
    integer i;
    always @(posedge clk) begin
        if (ce) begin
            for (i = 0; i < depth-1; i = i+1)
                regBank[i+1] <= regBank[i];
            regBank[0] <= din;
        end
    end
    assign dout = regBank[depth-1];
endmodule

```

for 循环形式的 32 位移位寄存器 (VHDL)

```

-- 32-bit shift register
-- Rising-edge clock
-- Active-high clock enable
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
entity gowin_shfitreg is
    generic(
        depth: integer := 32
    );
    port(

```

```
        clk : in std_logic;
        ce : in std_logic;
        din : in std_logic;
        dout : out std_logic
    );
end entity;

architecture Behavioral of gowin_shfitreg is
    signal regBank: std_logic_vector(depth-1 downto 0);
begin
    PROCESS(clk) begin
        IF(clk'event and clk = '1') then
            IF (ce = '1') then
                for i in 1 to depth-1 loop
                    regBank(i) <= regBank(i-1);
                end loop;
                regBank(0) <= din;
            end IF;
        end IF;
    end PROCESS;
    dout <= regBank(depth-1);
END Behavioral;
```

7.2 动态移位寄存器

动态移位寄存器是一种长度可在电路运行过程中动态变化的移位寄存器。

7.2.1 动态移位寄存器

动态移位寄存器通常包含以下元素：

- 时钟
- 可选时钟使能
- 串行数据输入
- 串行地址输入
- 串行数据输出

7.2.2 动态移位寄存器示例

位拼接形式的 32 位动态移位寄存器（Verilog）

```
// 32-bit dynamic shift register.
module gowin_shfitreg (clk, ce, din,addr, dout);
    parameter width_addr=5;
    parameter depth=2**width_addr;
    input clk, ce;
    input  din;
    input [width_addr-1:0] addr;
    output reg dout;
    reg dout_reg;
    reg [depth-1:0] regBank;
    always @(posedge clk) begin
        if (ce) begin
            regBank <= {regBank[depth-2:0],din};
        end
    end
    always @(posedge clk) begin
        if (ce) begin
            dout <= dout_reg;
            dout_reg<=regBank[addr];
        end
    end
endmodule
```

位拼接形式的 32 位动态移位寄存器（VHDL）

```
--32-bit dynamic shift register
Library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
entity gowin_shfitreg  is
    generic(
        width_addr: integer := 5;
        depth: integer := 2**width_addr
    );
```

```

    port(
        clk : in std_logic;
        ce : in std_logic;
        din : in std_logic;
        addr : in std_logic_vector(width_addr-1 downto 0);
        dout : out std_logic
    );
end entity;

architecture Behavioral of gowin_shfitreg is
    signal regBank: std_logic_vector(depth-1 downto 0);
    signal dout_reg : std_logic;
begin
    PROCESS(clk) begin
        IF(clk'event and clk = '1') then
            IF (ce = '1') then
                regBank <= regBank(depth-2 downto 0) & din;
            end IF;
        end IF;
    end PROCESS;

    PROCESS(clk) begin
        IF(clk'event and clk = '1') then
            IF (ce = '1') then
                dout <= dout_reg;
                dout_reg <= regBank(conv_integer(addr));
            end if;
        end IF;
    end PROCESS;

END Behavioral;

```

for 循环形式的 32 位动态移位寄存器（Verilog）

```

// 32-bit dynamic shift register
module gowin_shfitreg (clk, ce,addr, din, dout);
parameter addr_width=5;
parameter depth=2**addr_width;
input clk, ce;
input din;

```

```

input[addr_width-1:0]addr;
output reg dout;
reg dout_reg;
reg [depth-1:0]regBank;
integer i;
always @(posedge clk) begin
    if (ce) begin
        for (i = 0; i < depth-1; i = i+1)
            regBank[i+1] <= regBank[i];
        regBank[0] <= din;
    end
end
always @(posedge clk) begin
    if (ce) begin
        dout <=dout_reg;
        dout_reg<=regBank[addr];
    end
end
endmodule

```

for 循环形式的 32 位动态移位寄存器（VHDL）

```

-- 32-bit dynamic shift register
Library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
entity gowin_shfitreg is
    generic(
        width_addr: integer := 5;
        depth: integer := 2**width_addr
    );
    port(
        clk : in std_logic;
        ce : in std_logic;
        din : in std_logic;
        addr : in std_logic_vector(width_addr-1 downto 0);
        dout : out std_logic
    );
end entity

```

```
);  
end entity;  
architecture Behavioral of gowin_shfitreg is  
    signal regBank: std_logic_vector(depth-1 downto 0);  
    signal dout_reg : std_logic;  
begin  
    PROCESS(clk) begin  
        IF(clk'event and clk = '1') then  
            IF (ce = '1') then  
                for i in 1 to depth-1 loop  
                    regBank(i) <= regBank(i-1);  
                end loop;  
                regBank(0) <= din;  
            end IF;  
        end IF;  
    end PROCESS;  
    PROCESS(clk) begin  
        IF(clk'event and clk = '1') then  
            IF (ce = '1') then  
                dout <= dout_reg;  
                dout_reg<=regBank(conv_integer(addr));  
            end if;  
        end IF;  
    end PROCESS;  
END Behavioral;
```

8RAM

GowinSynthesis 支持各类 RAM 编写风格，并将其映射到分布式静态随机存储器 RAM (SSRAM) 或块状静态随机存储器 RAM (BSRAM)。

RAM 编码通常包含以下元素：

- 支持任意大小和数据宽度。GowinSynthesis 将 RAM 映射到一个或多个 RAM 原语，如单端口、伪双端口、双端口
- 最多两个写入端口
- 最多两个读取端口
- RAM 写使能 (BSRAM)
- 数据输出复位 (BSRAM)
- 可选输出寄存器 (BSRAM)
- 字节写入使能 (BSRAM)
- 每个 RAM 端口均可由其独特的时钟、端口使能、写入使能和数据输出复位控制
- RAM 初始化

注！

不要同时对同一地址的 RAM 进行读写操作。

8.1 块状静态随机存储器 (BSRAM)

8.1.1 单端口模式的 BSRAM 示例

写优先的单端口模式 BSRAM (Verilog)

```
// Single-Port Block RAM Write-First Mode
module gowin_ram (data_out, data_in, addr, clk, oce, ce, wre, rst);
    output reg [8:0] data_out;
    input [8:0] data_in;
    input [9:0] addr;
```

```
input clk,wre,oce,ce,rst;
reg [8:0] mem [1023:0];
reg [8:0]data_out_reg;

always@(posedge clk )begin
    if(rst)begin
        data_out <= 0;
    end
    else begin
        if(oce)begin
            data_out <= data_out_reg;
        end
    end
end

always@(posedge clk )begin
    if(rst)begin
        data_out_reg <= 0;
    end
    else begin
        if(ce & wre)begin
            data_out_reg <= data_in;
        end
        else begin
            if (ce & !wre)begin
                data_out_reg <= mem[addr];
            end
        end
    end
end

always @(posedge clk)begin
    if (ce & wre)begin
        mem[addr] <= data_in;
    end
end
```

```

endmodule

写优先的单端口模式 BSRAM (VHDL)

-- Single-Port Block RAM Write-First Mode
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
entity gowin_ram is
    generic(
        DATA_WIDTH : integer := 9;
        ADDRESS_WIDTH : integer := 10
    );
    port (
        data_in : in STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto
0);
        addr: in STD_LOGIC_VECTOR(ADDRESS_WIDTH - 1
downto 0);
        clk, oce, ce, wre,rst: in STD_LOGIC;
        data_out : out STD_LOGIC_VECTOR(DATA_WIDTH - 1
downto 0)
    );
end entity;

architecture rtl of gowin_ram is
    type mem_type is array (2**ADDRESS_WIDTH - 1 downto 0) of
STD_LOGIC_VECTOR (DATA_WIDTH - 1 downto 0);
    signal mem : mem_type;
    signal data_out_reg : STD_LOGIC_VECTOR(DATA_WIDTH - 1
downto 0);
begin
    process (clk)
    begin
        if (rising_edge(clk)) then
            if(ce = '1'and wre='1')then
                mem(conv_integer(addr)) <= data_in;
            end if;
        end if;
    end process;
end architecture;

```

```

end process;

process (clk)
begin
    if (rising_edge(clk)) then
        if(rst = '1')then
            data_out_reg<=(others=>'0');
        else
            if(ce = '1')then
                if(wre='1')then
                    data_out_reg <= data_in;
                else
                    data_out_reg <=
mem(conv_integer(addr));
                end if;
            end if;
        end if;
    end if;
end process;

process (clk)
begin
    if (rising_edge(clk)) then
        if(rst = '1')then
            data_out<=(others=>'0');
        else
            if(oce = '1')then
                data_out <= data_out_reg;
            end if;
        end if;
    end if;
end process;
end rtl;

```

读优先的单端口模式 BSRAM (Verilog)

// Single-Port Block RAM Read-First Mode

```
module gowin_ram(data_out, data_in, addr, clk, oce, ce, wre, rst);
output reg [8:0]data_out;
input [8:0]data_in;
input [9:0]addr;
input clk, wre, oce, ce, rst;
reg [8:0] mem [1023:0];
reg [8:0]data_out_reg;

always@(posedge clk )begin
    if(rst)begin
        data_out <= 0;
    end
    else begin
        if(oce)begin
            data_out <= data_out_reg;
        end
    end
end

always@(posedge clk )begin
    if(rst)begin
        data_out_reg <= 0;
    end
    else begin
        if(ce )begin
            data_out_reg <= mem[addr];
        end
    end
end

always @(posedge clk)begin
    if (ce & wre)begin
        mem[addr] <= data_in;
    end
end
endmodule
```

读优先的单端口模式 BSRAM (VHDL)

```

-- Single-Port Block RAM Read-First Mode
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
entity gowin_ram is
    generic(
        DATA_WIDTH : integer := 9;
        ADDRESS_WIDTH : integer := 10
    );
    port (
        data_in : in STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto
0);
        addr: in STD_LOGIC_VECTOR(ADDRESS_WIDTH - 1
downto 0);
        clk, oce, ce, wre,rst: in STD_LOGIC;
        data_out : out STD_LOGIC_VECTOR(DATA_WIDTH - 1
downto 0)
    );
end entity;

architecture rtl of gowin_ram is
    type mem_type is array (2**ADDRESS_WIDTH - 1 downto 0) of
        STD_LOGIC_VECTOR (DATA_WIDTH - 1 downto 0);
    signal mem : mem_type;
    signal data_out_reg : STD_LOGIC_VECTOR(DATA_WIDTH - 1
downto 0);
begin
    process (clk)
    begin
        if (rising_edge(clk)) then
            if(ce = '1'and wre='1')then
                mem(conv_integer(addr)) <= data_in;
            end if;
        end if;
    end process;
end architecture;

```

```

process (clk)
begin
    if (rising_edge(clk)) then
        if(rst = '1')then
            data_out_reg<=(others=>'0');
        else
            if(ce = '1')then
                data_out_reg <= mem(conv_integer(addr));
            end if;
        end if;
    end if;
end process;

```

```

process (clk)
begin
    if (rising_edge(clk)) then
        if(rst = '1')then
            data_out<=(others=>'0');
        else
            if(oce = '1')then
                data_out <= data_out_reg;
            end if;
        end if;
    end if;
end process;
end rtl;

```

Normal 模式的单端口模式 BSRAM (Verilog)

```

// Single-Port Block RAM Normal Mode
module gowin_ram (data_out, data_in, addr, clk,oce,ce, wre,rst);
output reg [8:0]data_out;
input [8:0]data_in;
input [9:0]addr;
input clk,wre,oce,ce,rst;
reg [8:0] mem [1023:0];

```

```

reg [8:0]data_out_reg;

always@(posedge clk )begin
    if(rst)begin
        data_out <= 0;
    end
    else begin
        if(oce)begin
            data_out <= data_out_reg;
        end
    end
end

always@(posedge clk )begin
    if(rst)begin
        data_out_reg <= 0;
    end
    else begin
        if(ce & !wre)begin
            data_out_reg <= mem[addr];
        end
    end
end

always @(posedge clk)begin
    if (ce & wre)begin
        mem[addr] <= data_in;
    end
end
endmodule

```

Normal 模式的单端口模式 BSRAM (VHDL)

```

-- Single-Port Block RAM Normal Mode
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
entity gowin_ram is

```

```

generic(
    DATA_WIDTH : integer := 9;
    ADDRESS_WIDTH : integer := 10
);
port (
    data_in : in STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto
0);
    addr: in STD_LOGIC_VECTOR(ADDRESS_WIDTH - 1
downto 0);
    clk, oce, ce, wre,rst: in STD_LOGIC;
    data_out : out STD_LOGIC_VECTOR(DATA_WIDTH - 1
downto 0)
);
end entity;

```

architecture rtl of gowin_ram is

```

    type mem_type is array (2**ADDRESS_WIDTH - 1 downto 0) of
STD_LOGIC_VECTOR (DATA_WIDTH - 1 downto 0);
    signal mem : mem_type;
    signal data_out_reg : STD_LOGIC_VECTOR(DATA_WIDTH - 1
downto 0);
begin
    process (clk)
    begin
        if (rising_edge(clk)) then
            if(ce = '1'and wre='1')then
                mem(conv_integer(addr)) <= data_in;
            end if;
        end if;
    end process;

    process (clk)
    begin
        if (rising_edge(clk)) then
            if(rst = '1')then
                data_out_reg<=(others=>'0');

```

```

        else
            if(ce = '1' and wre = '0')then
                data_out_reg <= mem(conv_integer(addr));
            end if;
        end if;
    end if;
end process;

process (clk)
begin
    if (rising_edge(clk)) then
        if(rst = '1')then
            data_out<=(others=>'0');
        else
            if(oce = '1')then
                data_out <= data_out_reg;
            end if;
        end if;
    end if;
end process;
end rtl;

```

8.1.2 伪双端口模式的 BSRAM 示例

单时钟伪双端口 BSRAM (Verilog)

```

// Semi Dual-Port Block RAM with One Clock
module gowin_ram(dout, din, ada, adb, clk, oce, cea, ceb, resetb);
    output reg[7:0]dout;
    input [7:0]din;
    input [9:0]ada, adb;
    input clk, oce, cea, ceb, resetb;
    reg [7:0] mem [1023:0];
    reg[7:0]dout_reg;
    always @(posedge clk)begin
        if (cea )begin
            mem[ada] <= din;

```

```

        end
    end

    always@(posedge clk)begin
        if(resetb)begin
            dout <= 0;
        end
        else if(oce)begin
            dout <= dout_reg;
        end
    end
end

always@(posedge clk)begin
    if(resetb)begin
        dout_reg <= 0;
    end
    else if(ceb)begin
        dout_reg <= mem[adb];
    end
end
endmodule

```

单时钟伪双端口 BSRAM (VHDL)

-- Semi Dual-Port Block RAM with One Clock

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

```

entity gowin_ram is

generic(

DATA_WIDTH : integer := 8;

ADDRESS_WIDTH : integer := 10

);

port (

din : in STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);

ada, adb: in STD_LOGIC_VECTOR(ADDRESS_WIDTH - 1

```

downto 0);
        clk, oce, cea, ceb, resetb: in STD_LOGIC;
        dout : out STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto
0));
    end entity;

```

architecture rtl of gowin_ram is

```

    type mem_type is array (2**ADDRESS_WIDTH - 1 downto 0) of
STD_LOGIC_VECTOR (DATA_WIDTH - 1 downto 0);
    signal mem : mem_type;
    signal dout_reg : STD_LOGIC_VECTOR(DATA_WIDTH - 1
downto 0);

```

begin

process(clk)

begin

if (rising_edge(clk)) then

if (resetb='1')then

dout <=(others=>'0');

else

if(oce = '1')then

dout <= dout_reg;

end if;

end if;

end if;

end process;

process(clk)

begin

if (rising_edge(clk)) then

if (resetb='1')then

dout_reg <=(others=>'0');

else

if(ceb = '1')then

dout_reg <= mem(conv_integer(adb));

end if;

```

        end if;
    end if;
end process;

process (clk)
begin
    if (rising_edge(clk)) then
        if(cea = '1')then
            mem(conv_integer(ada)) <= din;
        end if;
    end if;
end process;
end rtl;

```

双时钟伪双端口 BSRAM (Verilog)

```

// Semi Dual-Port Block RAM with two Clock
module gowin_ram(dout, din, ada, adb, clka, oce, cea, clkb, ceb,
resetb);
    output reg[7:0]dout;
    input [7:0]din;
    input [9:0]ada, adb;
    input clka, oce, cea, clkb, ceb, resetb;
    reg [7:0] mem [1023:0];
    reg[7:0]dout_reg;
    always @(posedge clka)begin
        if (cea )begin
            mem[ada] <= din;
        end
    end

    always@(posedge clkb)begin
        if(resetb)begin
            dout <= 0;
        end
        else if(oce)begin
            dout <= dout_reg;
        end
    end
end

```

```

        end
    end

    always@(posedge clkb)begin
        if(resetb)begin
            dout_reg <= 0;
        end
        else if(ceb)begin
            dout_reg <= mem[adb];
        end
    end
end
endmodule

```

双时钟伪双端口 BSRAM (VHDL)

-- Semi Dual-Port Block RAM with two Clock

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

```

```

entity gowin_ram is
    generic(
        DATA_WIDTH : integer := 8;
        ADDRESS_WIDTH : integer := 10
    );
    port (
        din : in STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);
        ada, adb: in STD_LOGIC_VECTOR(ADDRESS_WIDTH - 1
downto 0);
        clka, oce, cea, clkb, ceb, resetb: in STD_LOGIC;
        dout : out STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto
0));
end entity;

```

architecture rtl of gowin_ram is

```

    type mem_type is array (2**ADDRESS_WIDTH - 1 downto 0) of
        STD_LOGIC_VECTOR (DATA_WIDTH - 1 downto 0);

```

```
    signal mem : mem_type;
    signal dout_reg : STD_LOGIC_VECTOR(DATA_WIDTH - 1
downto 0);
begin
    process(clkb)
    begin
        if (rising_edge(clkb)) then
            if (resetb='1')then
                dout <=(others=>'0');
            else
                if(oce = '1')then
                    dout <= dout_reg;
                end if;
            end if;
        end if;
    end process;

    process(clkb)
    begin
        if (rising_edge(clkb)) then
            if (resetb='1')then
                dout_reg <=(others=>'0');
            else
                if(ceb = '1')then
                    dout_reg <= mem(conv_integer(adb));
                end if;
            end if;
        end if;
    end process;

    process (clka)
    begin
        if (rising_edge(clka)) then
            if(cea = '1')then
                mem(conv_integer(ada)) <= din;
```

```

        end if;
    end if;
end process;
end rtl;

```

不对称 RAM 示例

读位宽大于写位宽的不对称 RAM (Verilog)

```

// Asymmetric port RAM
// Read Wider than Write
module gowin_ram(dout, din, ada, clka, oce, cea, adb, clkb, ceb, rstb);
parameter adawidth = 8;
parameter diwidth = 6;
parameter adbwidth = 7;
parameter dowidth = 12;

output reg[dowidth-1:0]dout;
input [diwidth-1:0]din;
input [adawidth-1:0]ada;
input [adbwidth-1:0]adb;
input clka, cea, clkb, ceb, oce, rstb;
reg [diwidth-1:0]mem [2**adawidth-1:0];
reg [dowidth-1:0]dout_reg;
localparam b = 2**adawidth/2**adbwidth ;
integer j ;

always @(posedge clka)begin
    if (cea)begin
        mem[ada] <= din;
    end
end

always@(posedge clkb )begin
    if(rstb)begin
        dout_reg <= 0;
    end
    else begin

```

```

        if(ceb)begin
            for(j = 0;j < b;j = j+1)
                dout_reg[((j+1)*diwidth-1)-: diwidth]<=
mem[adb*b+j];
            end
        end
    end
    always@(posedge clkb )begin
        if(rstb)begin
            dout <= 0;
        end
        else begin
            if(oce)begin
                dout <= dout_reg;
            end
        end
    end
end
endmodule

```

读位宽大于写位宽的不对称 RAM (VHDL)

```

-- Asymmetric port RAM
-- Read Wider than Write
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
entity gowin_ram is
    generic(
        DATA_WIDTHB : integer := 12;
        ADDRESS_WIDTHB : integer := 7;
        ADDRESS_WIDTHA : integer := 8;
        DATA_WIDTHA : integer := 6
    );
    port (
        din : in STD_LOGIC_VECTOR(DATA_WIDTHA - 1 downto
0);
        ada: in STD_LOGIC_VECTOR(ADDRESS_WIDTHA - 1
downto 0);

```

```

        clka, cea, clk, ocb, rstb: in STD_LOGIC;
        adb: in STD_LOGIC_VECTOR(ADDRESS_WIDTHB - 1
downto 0);
        dout : out STD_LOGIC_VECTOR(DATA_WIDTHB - 1 downto
0)
        );
    end gowin_ram;

```

architecture rtl of gowin_ram is

```

        type mem_type is array (2**ADDRESS_WIDTHA - 1 downto 0) of
STD_LOGIC_VECTOR (DATA_WIDTHA - 1 downto 0);
        signal dout_reg : STD_LOGIC_VECTOR(DATA_WIDTHB - 1
downto 0);
        signal mem : mem_type;
    begin
        process (clka)
        begin
            if (rising_edge(clka)) then
                if(cea = '1')then
                    mem(conv_integer(ada)) <= din;
                end if;
            end if;
        end process;
        process(clkb)
        begin
            if (rising_edge(clkb)) then
                if (rstb='1')then
                    dout_reg <=(others=>'0');
                else
                    if(ceb = '1')then
                        dout_reg(DATA_WIDTHA*1 - 1 downto
DATA_WIDTHA*0) <= mem(conv_integer(adb&'0'));
                        dout_reg(DATA_WIDTHA*2 - 1 downto
DATA_WIDTHA*1) <= mem(conv_integer(adb&'1'));
                    end if;
                end if;
            end if;
        end process;
    end architecture rtl;

```

```

        end if;
    end process;
    process(clkb)
    begin
        if (rising_edge(clkb)) then
            if (rstb='1')then
                dout <=(others=>'0');
            else
                if(oce = '1')then
                    dout <= dout_reg;
                end if;
            end if;
        end if;
    end process;
end rtl;

```

写位宽大于读位宽的不对称 RAM (Verilog)

```

// Asymmetric port RAM
// Write wider than Read
module gowin_ram(dout, din, ada, clka, oce, cea, adb, clkb, ceb, rstb);
    parameter adawidth = 9;
    parameter diwidth = 16;
    parameter adbwidth = 10;
    parameter dowidth = 8;

    output reg[dowidth-1:0]dout;
    input [diwidth-1:0]din;
    input [adawidth-1:0]ada;
    input [adbwidth-1:0]adb;
    input clka, cea, clkb, ceb, oce, rstb;
    reg [dowidth-1:0]mem [2**adbwidth-1:0];
    reg [dowidth-1:0]dout_reg;

    always @(posedge clka)begin
        if (cea)begin
            mem[{ada, 1'b0}] <= din[dowidth*0+:dowidth];

```

```

        mem[{ada,1'b1}] <= din[dowidth*1+:dowidth];
    end
end

always@(posedge clkb )begin
    if(rstb)begin
        dout_reg <= 0;
    end
    else begin
        if(ceb)begin
            dout_reg<= mem[adb];
        end
    end
end
end
always@(posedge clkb )begin
    if(rstb)begin
        dout <= 0;
    end
    else begin
        if(oce)begin
            dout <= dout_reg;
        end
    end
end
end
endmodule

```

写位宽大于读位宽的不对称 RAM (VHDL)

```

-- Asymmetric port RAM
-- Write wider than Read
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
entity gowin_ram is
    generic(
        DATA_WIDTHB : integer := 8;
        ADDRESS_WIDTHB : integer := 10;

```

```

        ADDRESS_WIDTHA : integer := 9;
        DATA_WIDTHA : integer := 16
    );
    port (
        din : in STD_LOGIC_VECTOR(DATA_WIDTHA - 1 downto
0);
        ada: in STD_LOGIC_VECTOR(ADDRESS_WIDTHA - 1
downto 0);
        clka, cea, clkb, oce, ceb, rstb: in STD_LOGIC;
        adb: in STD_LOGIC_VECTOR(ADDRESS_WIDTHB - 1
downto 0);
        dout : out STD_LOGIC_VECTOR(DATA_WIDTHB - 1 downto
0)
    );
end entity;

```

architecture rtl of gowin_ram is

```

    type mem_type is array (2**ADDRESS_WIDTHB - 1 downto 0) of
STD_LOGIC_VECTOR (DATA_WIDTHB - 1 downto 0);
    signal dout_reg : STD_LOGIC_VECTOR(DATA_WIDTHB - 1
downto 0);
    signal mem : mem_type;
begin
    process (clka)
    begin
        if (rising_edge(clka)) then
            if(cea = '1')then
                mem(conv_integer(ada & '0')) <=
din(DATA_WIDTHB*1-1 downto DATA_WIDTHB*0);
                mem(conv_integer(ada & '1')) <=
din(DATA_WIDTHB*2-1 downto DATA_WIDTHB*1);
            end if;
        end if;
    end process;
    process(clkb)
    begin
        if (rising_edge(clkb)) then

```

```

        if (rstb='1')then
            dout_reg <=(others=>'0');
        else
            if(ceb = '1')then
                dout_reg<= mem(conv_integer(adb));
            end if;
        end if;
    end if;
end process;
process(clkb)
begin
    if (rising_edge(clkb)) then
        if (rstb='1')then
            dout <=(others=>'0');
        else
            if(oce = '1')then
                dout <= dout_reg;
            end if;
        end if;
    end if;
end process;
end rtl;

```

8.1.3 双端口模式的 BSRAM 示例

写优先的双端口模式 BSRAM (Verilog)

```

// Dual-Port Block RAM Write-First Mode
module gowin_ram(data_outa, data_ina, addra, clka, cea, ocea, wrea,
rsta, data_outb, data_inb, addrb, clk, ceb, oceb, wreb, rstb);
    output reg [7:0]data_outa,data_outb;
    input [7:0]data_ina,data_inb;
    input [9:0]addra,addrb;
    input clka,wrea,cea,ocea,rsta;
    input clk, wreb,ceb,oceb,rstb;
    reg [7:0] mem [1023:0];
    reg [7:0] data_outa_reg=16'h0000;
    reg [7:0] data_outb_reg=16'h0000;

```

```
always@(posedge clka)begin
    if(rsta)begin
        data_outa <= 0;
    end
    else begin
        if (oceas)begin
            data_outa <= data_outa_reg;
        end
    end
end
always@(posedge clka)begin
    if(rsta)begin
        data_outa_reg <= 0;
    end
    else begin
        if(ceas)begin
            if(!wreas)begin
                data_outa_reg <= mem[addra];
            end
            else begin
                data_outa_reg <= data_ina;
            end
        end
    end
end
always @(posedge clka)begin
    if (ceas & wreas) begin
        mem[addra] <= data_ina;
    end
end

always@(posedge clkb )begin
    if(rstb)begin
        data_outb <= 0;
    end
end
```

```

        else begin
            if (oceb)begin
                data_outb <= data_outb_reg;
            end
        end
    end
end
always@(posedge clkb )begin
    if(rstb)begin
        data_outb_reg <= 0;
    end
    else begin
        if(ceb)begin
            if(!wreb)begin
                data_outb_reg <= mem[addrb];
            end
            else begin
                data_outb_reg <= data_inb;
            end
        end
    end
end
end

always @(posedge clkb)begin
    if (ceb & wreb) begin
        mem[addrb] <= data_inb;
    end
end
endmodule

```

写优先的双端口模式 BSRAM (VHDL)

```

-- Dual-Port Block RAM Write-First Mode
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity gowin_ram is

```

```

generic(
    DATA_WIDTH : integer := 16;
    ADDRESS_WIDTH : integer := 9
);
port (
    data_ina, data_inb : in STD_LOGIC_VECTOR(DATA_WIDTH
- 1 downto 0);
    addra, addrb: in STD_LOGIC_VECTOR(ADDRESS_WIDTH
- 1 downto 0);
    clka, cea, ocea,wrea,rsta, clk, ceb, oceb,wreb,rstb: in
STD_LOGIC;
    data_outa, data_outb : out
STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0)
);
end entity;

```

architecture rtl of gowin_ram is

```

    type mem_type is array (2**ADDRESS_WIDTH - 1 downto 0) of
STD_LOGIC_VECTOR (DATA_WIDTH - 1 downto 0);
    signal mem : mem_type;
    signal data_outa_reg, data_outb_reg :
STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);
begin
    process(clka)
    begin
        if (rising_edge(clka)) then
            if(cea = '1' and wrea = '1') then
                mem(conv_integer(addra)) <= data_ina;
            end if;
        end if;
    end process;
    process(clka)
    begin
        if (rising_edge(clka)) then
            if(rsta='1')then
                data_outa_reg<=(others=>'0');
            end if;
        end if;
    end process;

```

```

        else
            if(cea = '1')then
                if(wrea = '0') then
                    data_outa_reg <=
mem(conv_integer(addr_a));
                else
                    data_outa_reg <= data_ina;
                end if;
            end if;
        end if;
    end process;
process(clka)
begin
    if (rising_edge(clka)) then
        if(rsta='1')then
            data_outa<=(others=>'0');
        else
            if(ocea = '1') then
                data_outa <= data_outa_reg;
            end if;
        end if;
    end if;
end process;

process (clkb)
begin
    if (rising_edge(clkb)) then
        if(rstb='1')then
            data_outb <= (others=>'0');
        else
            if(oceb = '1') then
                data_outb <= data_outb_reg;
            end if;
        end if;
    end if;
end process;

```

```

        end if;
    end process;
    process (clkb)
    begin
        if (rising_edge(clkb)) then
            if(rstb='1')then
                data_outb_reg <= (others=>'0');
            else
                if(ceb = '1')then
                    if(wreb = '0') then
                        data_outb_reg <=
mem(conv_integer(addrb));
                    else
                        data_outb_reg <= data_inb;
                    end if;
                end if;
            end if;
        end if;
    end process;
    process(clkb)
    begin
        if (rising_edge(clkb)) then
            if(ceb = '1' and wreb = '1') then
                mem(conv_integer(addrb)) <= data_inb;
            end if;
        end if;
    end process;
end rtl;

```

Normal 模式的双端口模式 BSRAM (Verilog)

```

// Dual-Port Block RAM Normal Mode
module gowin_ram(data_outa, data_ina, addra, clka, cea, ocea, wrea,
rsta, data_outb, data_inb, addrb, clkb, ceb, oceb, wreb, rstb);
    output reg [15:0]data_outa,data_outb;
    input [15:0]data_ina,data_inb;
    input [8:0]addra,addrb;

```

```
input clka,wrea,cea,ocea,rsta;
input clkb,wreb,ceb,oceb,rstb;
reg [15:0] mem [511:0];
reg [15:0] data_outa_reg=16'h0000;
reg [15:0] data_outb_reg=16'h0000;
always@(posedge clka)begin
    if(rsta)begin
        data_outa <= 0;
    end
    else begin
        if (ocea)begin
            data_outa <= data_outa_reg;
        end
    end
end
always@(posedge clka)begin
    if(rsta)begin
        data_outa_reg <= 0;
    end
    else begin
        if(cea & !wrea)begin
            data_outa_reg <= mem[addra];
        end
    end
end
always @(posedge clka)begin
    if (cea & wrea) begin
        mem[addra] <= data_ina;
    end
end

always@(posedge clkb )begin
    if(rstb)begin
        data_outb <= 0;
    end
end
```

```

        else begin
            if (oceb)begin
                data_outb <= data_outb_reg;
            end
        end
    end
end
always@(posedge clkb )begin
    if(rstb)begin
        data_outb_reg <= 0;
    end
    else begin
        if(ceb & !wreb)begin
            data_outb_reg <= mem[addrb];
        end
    end
end
end

always @(posedge clkb)begin
    if (ceb & wreb) begin
        mem[addrb] <= data_inb;
    end
end
endmodule

```

Normal 模式的双端口模式 BSRAM (VHDL)

```

-- Dual-Port Block RAM Normal Mode
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity gowin_ram is
    generic(
        DATA_WIDTH : integer := 16;
        ADDRESS_WIDTH : integer := 9
    );
    port (

```

```

        data_ina, data_inb : in STD_LOGIC_VECTOR(DATA_WIDTH
- 1 downto 0);
        addra, addrb: in STD_LOGIC_VECTOR(ADDRESS_WIDTH
- 1 downto 0);
        clka, cea, ocea,wrea,rsta, clk b, ceb, oceb,wreb,rstb: in
STD_LOGIC;
        data_outa, data_outb : out
STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0)
    );
end entity;

```

architecture rtl of gowin_ram is

```

    type mem_type is array (2**ADDRESS_WIDTH - 1 downto 0) of
STD_LOGIC_VECTOR (DATA_WIDTH - 1 downto 0);
    signal mem : mem_type;
    signal data_outa_reg, data_outb_reg :
STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);
begin
    process(clka)
    begin
        if (rising_edge(clka)) then
            if(cea = '1' and wrea = '1') then
                mem(conv_integer(addra)) <= data_ina;
            end if;
        end if;
    end process;
    process(clka)
    begin
        if (rising_edge(clka)) then
            if(rsta='1')then
                data_outa_reg<=(others=>'0');
            else
                if(cea = '1' and wrea = '0') then
                    data_outa_reg <= mem(conv_integer(addra));
                end if;
            end if;
        end if;
    end process;
end rtl;

```

```

        end if;
    end process;
    process(clka)
    begin
        if (rising_edge(clka)) then
            if(rsta='1')then
                data_outa<=(others=>'0');
            else
                if(ocea = '1') then
                    data_outa <= data_outa_reg;
                end if;
            end if;
        end if;
    end process;

    process (clkb)
    begin
        if (rising_edge(clkb)) then
            if(rstb='1')then
                data_outb <= (others=>'0');
            else
                if(oceb = '1') then
                    data_outb <= data_outb_reg;
                end if;
            end if;
        end if;
    end process;
    process (clkb)
    begin
        if (rising_edge(clkb)) then
            if(rstb='1')then
                data_outb_reg <= (others=>'0');
            else
                if(ceb = '1' and wreb = '0') then
                    data_outb_reg <= mem(conv_integer(addrb));
                end if;
            end if;
        end if;
    end process;

```

```

        end if;
    end if;
end if;
end process;
process(clkb)
begin
    if (rising_edge(clkb)) then
        if(ceb = '1' and wreb = '1') then
            mem(conv_integer(addrb)) <= data_inb;
        end if;
    end if;
end process;
end rtl;

```

8.1.4 Byte-enable 功能

Byte-enable 功能—读优先的单端口模式 BSRAM (Verilog)

```

// Single-port BSRAM with Byte-wide Write Enable
// Read-First mode
module gowin_ram (dout, clk, oce, ce, reset, wre, ad, din, byte_en);
output reg[35:0] dout;
input clk,oce,ce,reset,wre;
input [7:0] ad;
input [35:0] din;
input [3:0] byte_en;

reg[35:0]mem[2**8-1:0];
reg[35:0] dout_reg;
always@(posedge clk)begin
    if(ce & wre)begin
        if(byte_en[0])begin
            mem[ad][8:0]<=din[8:0];
        end
        if(byte_en[1])begin
            mem[ad][17:9]<=din[17:9];
        end
    end
end

```

```

        if(byte_en[2])begin
            mem[ad][26:18]<=din[26:18];
        end
        if(byte_en[3])begin
            mem[ad][35:27]<=din[35:27];
        end
    end
end

```

```

always@(posedge clk)begin
    if(reset)begin
        dout_reg<=0;
    end
    else begin
        if(ce)begin
            dout_reg<=mem[ad];
        end
    end
end

```

```

always@(posedge clk)begin
    if(reset)begin
        dout<=0;
    end
    else begin
        if(oce)begin
            dout<=dout_reg;
        end
    end
end
endmodule

```

Byte-enable 功能 – 读优先的单端口模式 BSRAM (VHDL)

```

-- Single-port BSRAM with Byte-wide Write Enable
-- Read-First mode
library IEEE;

```

```

use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity gowin_ram is
    generic(
        DATA_WIDTH : integer := 36;
        ADDRESS_WIDTH : integer := 8
    );
    port (
        din : in STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);
        ad: in STD_LOGIC_VECTOR(ADDRESS_WIDTH - 1 downto
0);

        clk,oce,ce, wre,reset: in STD_LOGIC;
        byte_en : in STD_LOGIC_VECTOR(3 downto 0);
        dout : out STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto
0)
    );
end entity;

architecture rtl of gowin_ram is
    type mem_type is array (2**ADDRESS_WIDTH - 1 downto 0) of
        STD_LOGIC_VECTOR (DATA_WIDTH - 1 downto 0);
    signal mem : mem_type ;
    signal dout_reg : STD_LOGIC_VECTOR(DATA_WIDTH - 1
downto 0);
begin
    process (clk)
    begin
        if (rising_edge(clk)) then
            if(reset='1')then
                dout <=(others=>'0');
            else
                if(oce ='1')then
                    dout <= dout_reg;
                end if;
            end if;
        end if;
    end process;
end architecture;

```

```

        end if;
    end process;

    process (clk)
    begin
        if (rising_edge(clk)) then
            if(reset='1')then
                dout_reg <=(others=>'0');
            else
                if(ce ='1')then
                    dout_reg <= mem(conv_integer(ad));
                end if;
            end if;
        end if;
    end process;

    process (clk)
    begin
        if (rising_edge(clk)) then
            if(ce ='1'and wre = '1')then
                if(byte_en(0)='1')then
                    mem(conv_integer(ad))(8 downto 0) <= din(8
downto 0);

                    end if;
                    if(byte_en(1)='1')then
                        mem(conv_integer(ad))(17 downto 9) <= din(17
downto 9);

                        end if;
                        if(byte_en(2)='1')then
                            mem(conv_integer(ad))(26 downto 18) <=
din(26 downto 18);

                            end if;
                            if(byte_en(3)='1')then
                                mem(conv_integer(ad))(35 downto 27) <=
din(35 downto 27);

                                end if;

```

```

        end if;
    end if;
end process;
end rtl;

```

Byte-enable 功能 – 写优先的双端口模式 BSRAM (Verilog)

```

// Dual-Port BRAM with Byte-wide Write Enable
// Write First mode
module gowin_ram (douta, doutb, clka, ocea, cea, reseta, wrea,
clkb, oceb, ceb, resetb, wreb, ada, dina, adb, dinb, byte_ena, byte_enb);
    output reg [15:0] douta;
    output reg [15:0] doutb;
    input clka, ocea, cea, reseta, wrea;
    input clkb, oceb, ceb, resetb, wreb;
    input [8:0] ada, adb;
    input [15:0] dina, dinb;
    input [1:0] byte_ena, byte_enb;

    reg[15:0] mem[2**9-1:0];
    reg [15:0] douta_reg, doutb_reg;

    always@(posedge clka)begin
        if(cea & wrea)begin
            if(byte_ena[0])begin
                mem[ada][7:0]<=dina[7:0];
            end
            if(byte_ena[1])begin
                mem[ada][15:8]<=dina[15:8];
            end
        end
    end

    always@(posedge clka)begin
        if(reseta)begin
            douta_reg<=0;
        end
    end

```

```
    else begin
        if(cea)begin
            if(!wrea)begin
                douta_reg<=mem[ada];
            end
        else begin
            if(byte_ena[0])begin
                douta_reg[7:0]<=dina[7:0];
            end
            else begin
                douta_reg[7:0]<=mem[ada][7:0];
            end
            if(byte_ena[1])begin
                douta_reg[15:8]<=dina[15:8];
            end
            else begin
                douta_reg[15:8]<=mem[ada][15:8];
            end
        end
    end
end

always@(posedge clka)begin
    if(reseta)begin
        douta<=0;
    end
    else begin
        if(ocea)begin
            douta<=douta_reg;
        end
    end
end
```

```
always@(posedge clkb)begin
    if(ceb & wreb)begin
        if(byte_enb[0])begin
            mem[adb][7:0]<=dinb[7:0];
        end
        if(byte_enb[1])begin
            mem[adb][15:8]<=dinb[15:8];
        end
    end
end

always@(posedge clkb)begin
    if(reseta)begin
        doutb_reg<=0;
    end
    else begin
        if(ceb)begin
            if(!wreb)begin
                doutb_reg<=mem[adb];
            end
            else begin
                if(byte_enb[0])begin
                    doutb_reg[7:0]<=dinb[7:0];
                end
                else begin
                    doutb_reg[7:0]<=mem[adb][7:0];
                end
                if(byte_enb[1])begin
                    doutb_reg[15:8]<=dinb[15:8];
                end
                else begin
                    doutb_reg[15:8]<=mem[adb][15:8];
                end
            end
        end
    end
end
```

```

        end
    end

    always@(posedge clkb)begin
        if(resetb)begin
            doutb<=0;
        end
        else begin
            if(oceb)begin
                doutb<=doutb_reg;
            end
        end
    end
end
endmodule

```

Byte-enable 功能—写优先的双端口模式 BSRAM (VHDL)

```

-- Dual-Port BRAM with Byte-wide Write Enable
-- Write First mode
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity gowin_ram is
    generic(
        DATA_WIDTH : integer := 16;
        ADDRESS_WIDTH : integer := 9
    );
    port (
        dina, dinb : in STD_LOGIC_VECTOR(DATA_WIDTH - 1
downto 0);
        ada, adb: in STD_LOGIC_VECTOR(ADDRESS_WIDTH - 1
downto 0);
        clka, ocea, cea, wrea, reseta, clkb, oceb, ceb, wreb, resetb: in
STD_LOGIC;
        byte_ena, byte_enb : in STD_LOGIC_VECTOR(1 downto 0);
        douta, doutb : out STD_LOGIC_VECTOR(DATA_WIDTH - 1
downto 0)
    );
end entity gowin_ram;

```

```

    );
end entity;

architecture rtl of gowin_ram is
    type mem_type is array (2**ADDRESS_WIDTH - 1 downto 0) of
        STD_LOGIC_VECTOR (DATA_WIDTH - 1 downto 0);
    signal mem : mem_type;
    signal douta_reg, doutb_reg :
        STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);
begin

    process (clka)
    begin
        if (rising_edge(clka)) then
            if(reseta='1')then
                douta <=(others=>'0');
            else
                if(ocea ='1')then
                    douta <= douta_reg;
                end if;
            end if;
        end if;
    end process;

    process (clka)
    begin
        if (rising_edge(clka)) then
            if(reseta='1')then
                douta_reg <=(others=>'0');
            else
                if(cea ='1')then
                    if(wrea = '0')then
                        douta_reg <= mem(conv_integer(ada));
                    else
                        if(byte_ena(0)='1')then

```



```

        if (rising_edge(clkb)) then
            if(resetb='1')then
                doutb <=(others=>'0');
            else
                if(oceb ='1')then
                    doutb <= doutb_reg;
                end if;
            end if;
        end if;
    end process;

    process (clkb)
    begin
        if (rising_edge(clkb)) then
            if(resetb='1')then
                doutb_reg <=(others=>'0');
            else
                if(ceb ='1')then
                    if(wreb = '0')then
                        doutb_reg <= mem(conv_integer(adb));
                    else
                        if(byte_enb(0)='1')then
                            doutb_reg(7 downto 0) <= dinb(7
downto 0);

                                else
                                    doutb_reg(7 downto 0) <=
mem(conv_integer(adb))(7 downto 0);
                                end if;
                                if(byte_enb(1)='1')then
                                    doutb_reg(15 downto 8) <= dinb(15
downto 8);

                                        else
                                            doutb_reg(15 downto 8) <=
mem(conv_integer(adb))(15 downto 8);
                                        end if;
                                    end if;
                                end if;
                            end if;
                        end if;
                    end if;
                end if;
            end if;
        end if;
    end process;

```

```

        end if;
    end if;
end if;
end process;

process (clkb)
begin
    if (rising_edge(clkb)) then
        if(ceb ='1'and wreb = '1')then
            if(byte_enb(0)='1')then
                mem(conv_integer(adb))(7 downto 0) <= dinb(7
downto 0);
            end if;
            if(byte_enb(1)='1')then
                mem(conv_integer(adb))(15 downto 8) <=
dinb(15 downto 8);
            end if;
        end if;
    end process;

end rtl;

```

Byte-enable 功能 – Normal 模式的伪双端口 BSRAM (Verilog)

```

// Semi Dual-Port BRAM with Byte-wide Write Enable
// Normal mode
module gowin_ram (dout, clka, cea, clkb, oce,ceb, reset, ada, din, adb,
byte_ena);
    output reg[31:0] dout;
    input clka,cea, clkb,ceb,reset,oce;
    input [7:0] ada,adb;
    input [31:0] din;
    input [3:0] byte_ena;

    reg[31:0]mem[2**8-1:0];
    reg[31:0] dout_reg;

```

```
always@(posedge clka)begin
    if(cea)begin
        if(byte_ena[0])begin
            mem[ada][7:0]<=din[7:0];
        end
        if(byte_ena[1])begin
            mem[ada][15:8]<=din[15:8];
        end
        if(byte_ena[2])begin
            mem[ada][23:16]<=din[23:16];
        end
        if(byte_ena[3])begin
            mem[ada][31:24]<=din[31:24];
        end
    end
end

always@(posedge clkb)begin
    if(reset)begin
        dout_reg<=0;
    end
    else begin
        if(ceb)begin
            dout_reg<=mem[adb];
        end
    end
end

always@(posedge clkb)begin
    if(reset)begin
        dout<=0;
    end
    else begin
        if(oce)begin
            dout<=dout_reg;
        end
    end
end
```

```

        end
    end
endmodule

```

Byte-enable 功能—Normal 模式的伪双端口 BSRAM (VHDL)

```

-- Semi Dual-Port BRAM with Byte-wide Write Enable
-- Normal mode
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
entity gowin_ram is
    generic(
        ADDRESS_WIDTH : integer := 8;
        DATA_WIDTH : integer := 32
    );
    port (
        din : in STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);
        ada : in STD_LOGIC_VECTOR(ADDRESS_WIDTH - 1 downto 0);
        clka, cea, clkb, oce,ceb,reset: in STD_LOGIC;
        adb: in STD_LOGIC_VECTOR(ADDRESS_WIDTH - 1 downto 0);
        byte_ena : in STD_LOGIC_VECTOR(3 downto 0);
        dout : out STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0));
end gowin_ram;
architecture rtl of gowin_ram is
    type mem_type is array (2**ADDRESS_WIDTH - 1 downto 0) of
STD_LOGIC_VECTOR (DATA_WIDTH - 1 downto 0);
    signal mem : mem_type;
    signal dout_reg : STD_LOGIC_VECTOR(DATA_WIDTH - 1
downto 0);
begin
    process(clkb)
    begin
        if (rising_edge(clkb)) then
            if (reset='1')then
                dout <=(others=>'0');
            else

```

```

        if(oce = '1')then
            dout <= dout_reg;
        end if;
    end if;
end process;

process(clkb)
begin
    if (rising_edge(clkb)) then
        if (reset='1')then
            dout_reg <=(others=>'0');
        else
            if(ceb = '1')then
                dout_reg <= mem(conv_integer(adb));
            end if;
        end if;
    end if;
end process;

process (clka)
begin
    if (rising_edge(clka)) then
        if(cea = '1')then
            if(byte_ena(0)='1')then
                mem(conv_integer(ada))(7 downto 0) <= din(7
downto 0);
            end if;

            if(byte_ena(1)='1')then
                mem(conv_integer(ada))(15 downto 8) <=
din(15 downto 8);
            end if;
            if(byte_ena(2)='1')then
                mem(conv_integer(ada))(23 downto 16) <=
din(23 downto 16);

```

```

        end if;
        if(byte_ena(3)='1')then
            mem(conv_integer(ada))(31 downto 24) <=
din(31 downto 24);
        end if;
    end if;
end if;
end process;
end rtl;

```

8.2 分布式静态随机存储器 (SSRAM)

8.2.1 具有同步读取功能的单端口模式 SSRAM 示例

具有同步读取功能的单端口模式 SSRAM (Verilog)

```

// Single-Port RAM with Synchronous Read (Distributed RAM)
module gowin_ram (data_out, data_in, addr, clk, wre,rst);
    output reg[3:0]data_out;
    input [3:0]data_in;
    input [3:0]addr;
    input clk,wre,rst;
    reg [3:0] mem [15:0];
    always @(posedge clk)begin
        if ( wre) begin
            mem[addr] <= data_in;
        end
    end
    always @(posedge clk)begin
        if(rst)begin
            data_out<=0;
        end
        else begin
            data_out <= mem[addr];
        end
    end
end
endmodule

```

具有同步读取功能的单端口模式的 SSRAM (VHDL)

-- Single-Port RAM with Synchronous Read (Distributed RAM)

library IEEE;

use IEEE.std_logic_1164.all;

use IEEE.std_logic_unsigned.all;

entity gowin_ram is

generic(

DATA_WIDTH : integer := 4;

ADDRESS_WIDTH : integer := 4

);

port (

data_in : in STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);

addr: in STD_LOGIC_VECTOR(ADDRESS_WIDTH - 1 downto 0);

clk, wre,rst: in STD_LOGIC;

data_out : out STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0)

);

end gowin_ram;

architecture rtl of gowin_ram is

type mem_type is array (2**ADDRESS_WIDTH - 1 downto 0) of
STD_LOGIC_VECTOR (DATA_WIDTH - 1 downto 0);

signal mem : mem_type;

begin

process (clk)

begin

if (rising_edge(clk)) then

if(wre = '1')then

mem(conv_integer(addr)) <= data_in;

end if;

```

        end if;
    end process;

    process (clk)
    begin
        if (rising_edge(clk)) then
            if( rst = '1')then
                data_out <= (others=>'0');
            else
                data_out <= mem(conv_integer(addr));
            end if;
        end if;
    end process;
end rtl;

```

8.2.2 具有同步读取功能的伪双端口模式的 SSRAM 示例

具有同步读取功能的伪双端口模式 SSRAM (Verilog)

```

// Semi Dual-Port RAM with Synchronous Read (Distributed RAM)
module gowin_ram (data_out, data_in, addra, clk, wre, rst,addrb);
    output reg [3:0]data_out;
    input [3:0]data_in;
    input [3:0] addra ,addrb;
    input clk,wre,rst;

    reg [3:0] mem [15:0];

    always @(posedge clk)begin
        if (wre)begin
            mem[addra] <= data_in;
        end
    end
    always @(posedge clk)begin
        if (rst)begin
            data_out<=0;
        end
    end
end

```

```

        else begin
            data_out <= mem[addrb];
        end
    end
endmodule

```

具有同步读取功能的伪双端口模式 SSRAM (VHDL)

-- Semi Dual-Port RAM with Synchronous Read (Distributed RAM)

```
library IEEE;
```

```
use IEEE.std_logic_1164.all;
```

```
use IEEE.std_logic_unsigned.all;
```

```
entity gowin_ram is
```

```
    generic(
```

```
        DATA_WIDTH : integer := 4;
```

```
        ADDRESS_WIDTH : integer := 4
```

```
    );
```

```
    port (
```

```
        data_in : in STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);
```

```
        ada, adb: in STD_LOGIC_VECTOR(ADDRESS_WIDTH - 1 downto 0);
```

```
        clk, wre,rst: in STD_LOGIC;
```

```
        data_out : out STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0));
```

```
end gowin_ram;
```

```
architecture rtl of gowin_ram is
```

```
    type mem_type is array (2**ADDRESS_WIDTH - 1 downto 0) of
        STD_LOGIC_VECTOR (DATA_WIDTH - 1 downto 0);
```

```
    signal mem : mem_type;
```

```
begin
```

```
    process (clk)
```

```
    begin
```

```
        if (rising_edge(clk)) then
```

```
            if(wre = '1')then
```

```

        mem(conv_integer(ada)) <= data_in;
    end if;
end if;
end process;

process (clk)
begin
    if (rising_edge(clk)) then
        if(rst = '1')then
            data_out <=(others=>'0');
        else
            data_out <= mem(conv_integer(adb));
        end if;
    end if;
end process;

end rtl;

```

8.3 RAM 初始化

RAM 可以通过以下方式初始化：

1. 在 HDL 源代码中指定 RAM 初始化内容。
2. 在外部数据文件中指定 RAM 初始化内容。

8.3.1 HDL 源代码中指定 RAM 初始化内容

RAM 定义时赋初值 (Verilog)

```

// Initializing Block RAM (Distributed RAM)
module gowin_ram (data_out, data_in, addra, addrb, clk, ce,rst);
    output reg[15:0]data_out;
    input [15:0]data_in;
    input [2:0]addra, addrb;
    input clk, ce,rst;
    reg [15:0] mem [7:0]={16'h0123,16'h4567,16'h89ab,16'hcdef,
16'h0147, 16'h0258,16'h789a,16'h5678};

    always @(posedge clk)begin
        if (ce)begin

```

```

        mem[addra] <= data_in;
    end
end
always @(posedge clk)begin
    if (rst)begin
        data_out<=0;
    end
    else begin
        data_out<= mem[addrb];
    end
end
end
endmodule

```

RAM 定义时赋初值 (VHDL)

```

-- Initializing Block RAM (Distributed RAM)
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
entity gowin_ram is
    generic(
        DATA_WIDTH : integer := 16;
        ADDRESS_WIDTH : integer := 3
    );
    port (
        data_in : in STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto
0);
        addra, addrb: in STD_LOGIC_VECTOR(ADDRESS_WIDTH
- 1 downto 0);
        clk,ce,rst: in STD_LOGIC;
        data_out : out STD_LOGIC_VECTOR(DATA_WIDTH - 1
downto 0));
    end gowin_ram;
    architecture rtl of gowin_ram is
        type mem_type is array (2**ADDRESS_WIDTH - 1 downto 0) of
STD_LOGIC_VECTOR (DATA_WIDTH - 1 downto 0);
        signal mem :
mem_type:=("0000000100100011","0100010101100111","1000100110101

```

```

011","1100110111101111","0000000101000111","0000001001011000","011
1100010011010","0101011001111000");

    signal dout_reg : STD_LOGIC_VECTOR(DATA_WIDTH - 1
downto 0);
begin
    process (clk)
    begin
        if (rising_edge(clk)) then
            if(ce = '1')then
                mem(conv_integer(addr_a)) <= data_in;
            end if;
        end if;
    end process;

    process (clk)
    begin
        if (rising_edge(clk)) then
            if(rst = '1')then
                data_out <= (others=>'0');
            else
                data_out <= mem(conv_integer(addr_b));
            end if;
        end if;
    end process;
end rtl;

```

Initial 块给 RAM 赋初值 (Verilog)

```

// Initializing Block RAM (Semi Dual-Port Block RAM)
module gowin_ram (dout, din, ada, adb, clka, oce, cea, clkb, ceb,
resetb);
    output reg[15:0]dout;
    input [15:0]din;
    input [8:0]ada, adb;
    input clka, oce, cea, clkb, ceb, resetb;
    reg [15:0] mem [511:0];
    reg[15:0]dout_reg;

```

```

integer i;
initial begin
    for (i=0; i<512; i=i+1)
        mem[i] = i;
end
always @(posedge clka)begin
    if (cea )begin
        mem[ada] <= din;
    end
end

always@(posedge clkb)begin
    if(resetb)begin
        dout <= 0;
    end
    else if(oce)begin
        dout <= dout_reg;
    end
end

always@(posedge clkb)begin
    if(resetb)begin
        dout_reg <= 0;
    end
    else if(ceb)begin
        dout_reg <= mem[adb];
    end
end
endmodule

```

8.3.2 外部数据文件中指定 RAM 初始化内容

readmemh 给 RAM 赋初值 (Verilog)

```

// Initializing Block RAM from external data file
// Hexadecimal data
module gowin_ram (data_out, data_in, addra, addrb, clk, ce,rst);

```

```
output reg[15:0]data_out;
input [15:0]data_in;
input [2:0]addra, addrb;
input clk, ce,rst;
reg [15:0] mem [7:0];

initial begin
    $readmemh("E:/ram.mi", mem);
end

always @(posedge clk)begin
    if (ce)begin
        mem[addra] <= data_in;
    end
end

always @(posedge clk)begin
    if (rst)begin
        data_out<=0;
    end
    else begin
        data_out<= mem[addrb];
    end
end

endmodule
```

ram.mi 数据书写形式如下:

```
0123
4567
89ab
cdef
0147
0258
789a
5678
```

readmemb 给 RAM 赋初值 (Verilog)

```
// Initializing Block RAM from external data file
// Binary data
module gowin_ram (data_out, data_in, addra, addrb, clk, ce,rst);
output reg[15:0]data_out;
input [15:0]data_in;
input [2:0]addra, addrb;
input clk, ce,rst;
reg [15:0] mem [7:0];

initial begin
    $readmemb("E:/ram.mi", mem);
end

always @(posedge clk)begin
    if (ce)begin
        mem[addra] <= data_in;
    end
end

always @(posedge clk)begin
    if (rst)begin
        data_out<=0;
    end
    else begin
        data_out<= mem[addrb];
    end
end
endmodule
```

ram.mi 数据书写形式如下:

```
0000000100100011
0100010110110011
1000100110101011
1101011010001011
```

```

0000000101010101
1111011101110111
1110010110110011
1001100011100000

```

readline 给 RAM 赋初值 (VHDL)

```

-- Initializing Block RAM from external data file
-- Binary data
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use std.textio.all;
entity gowin_ram is
    port(
        clk : in std_logic;
        we : in std_logic;
        addra,addrb : in std_logic_vector(5 downto 0);
        din : in std_logic_vector(31 downto 0);
        dout : out std_logic_vector(31 downto 0)
    );
end gowin_ram;
architecture syn of gowin_ram is
    type RamType is array (0 to 63) of bit_vector(31 downto 0);
    impure function InitRam(RamFileName : in string) return
RamType is
        FILE RamFile : text is in RamFileName;
        variable RamFileLine : line;
        variable RAM : RamType;
        begin
            for i in RamType'range loop
                readline(RamFile, RamFileLine);
                read(RamFileLine, RAM(i));
            end loop;
            return RAM;
        end function;
    signal RAM : RamType := InitRam("E:/ram.mi");

```

```

begin
process(clk)
begin
    if clk'event and clk = '1' then
        if we = '1' then
            RAM(to_integer(unsigned(addr_a))) <= to_bitvector(din);
        end if;
        dout <=
to_stdlogicvector(RAM(to_integer(unsigned(addr_b))));
    end if;
end process;
end syn;

```

ram.mi 数据书写形式如下:

```

00000001001000111111011101110111
01000101101100110000000100100011
1000100110101011101011010001011
11010110100010111110110110110011
10000001010101011101011010001011
1111011101101111100111011011001
11100101101100111100111011011001
10011000111000001000100110101011
11000001001000111110011101110111
11000101101100111110010110110011
10101001101010110110100110101011
10010110100010111110011101110111
10100001010101011001100011100000
11100111011101111010000101010101
11101101101100110110100110101011
10010000111000001110011101110111
11000010010001101100000100100011
01000101101100101000100110101011
01101001101010111110010110110011
11010110100010111001100011100000
01100001010101011110011101110111
11001110110110010110000101010101

```

```
10011000111000001111011101100111
11000001001000110000000101010101
11000101101100111000100110101011
10101001101010111011011101110111
10010110100010111111000011111111
10100001010101011111000011011011
11100111011101111111000011110011
11101101101100111111000011111111
10010000111111111110010110100011
00000001001000111111000011111111
00000001001000111111011101110111
01000101101100110000000100100011
10001001101010111101011010001011
11010110100010111110110110110011
10000001010101011101011010001011
11110111011101111100111011011001
11100101101100111100111011011001
10011000111000001000100110101011
11000001001000111110011101110111
11000101101100111110010110110011
10101001101010110110100110101011
10010110100010111110011101110111
10100001010101011001100011100000
11100111011101111010000101010101
11101101101100110110100110101011
10010000111000001110011101110111
11000010010001101100000100100011
01000101101100101000100110101011
01101001101010111110010110110011
11010110100010111001100011100000
01100001010101011110011101110111
11001110110110010110000101010101
10011000111000001111011101100111
11000001001000110000000101010101
11000101101100111000100110101011
```

```
10101001101010111011011101110111
10010110100010111111000011111111
10100001010101011111000011011011
11100111011101111111000011110011
11101101101100111111000011111111
100100001111111111110010110100011
00000001001000111111000011111111
```

9_{ROM}

只读存储器 (ROM) 与随机存取存储器 (RAM) 非常相似。

9.1 ROM 示例

9.1.1 定义时赋初值的 ROM (Verilog)

```
// ROM Inference on array
module gowin_rom(dout, addr,clk,oce, ce,rst);
output reg[31:0]dout;
input [5:0]addr;
input clk, ce,oce,rst;
reg[31:0]dout_reg;
    reg [31:0] mem [63:0]={32'h52853fd5,32'h38581bd2, 32'h040d53e4,
32'h22ce7d00,32'hc0b4bf1c,32'hec45e626, 32'hd9d000d9, 32'haacf8574,
        32'h8c565693,32'hb19808d0, 32'he073036e,
32'h41b923f6,32'hba17fce1,32'hd4dec5de, 32'ha18ad699, 32'h4a734008,
        32'h8f26bdd4,32'hb8d4aab6, 32'hf55e3c77,
32'h41a5d418,32'h5c651d69,32'h445469c3, 32'h2e49668b,
32'hdc1aa05b,
        32'h1e1f0f1e,32'h86fd31ef, 32'h73d90e02,
32'hb655bf16,32'hdce89022,32'h5c32ac0e, 32'hba172648, 32'hcebfe4cd,
        32'h32853fd5,32'h3858abd2, 32'h04ac53e4,
32'h22ce7d00,32'hc0b4bf1c,32'hec45e626, 32'hd9d000d9, 32'haacf8574,
        32'h8c565693,32'hb19808d0, 32'he073036e,
32'h41b923f6,32'hba17fce1,32'hd4dec5de, 32'ha18ad699, 32'h4da34008,
        32'h8f26bdd4,32'hb8d4aab6, 32'hf55e3c77,
32'h41a5d418,32'h5c651d69,32'h445469c3, 32'h2e49668b,
32'hdc1aa05b,
        32'h1e1f0f1e,32'h86fd31ef, 32'h73d90e02,
32'hb655bf16,32'hdce89022,32'h5c32ac0e, 32'hba172648, 32'hcebfe4cd
```

```

};
always @(posedge clk)begin
    if (rst)begin
        dout<=0;
    end
    else begin
        if(oce)begin
            dout<= dout_reg;
        end
    end
end
always @(posedge clk)begin
    if (rst)begin
        dout_reg<=0;
    end
    else begin
        if(ce)begin
            dout_reg<= mem[addr];
        end
    end
end
endmodule

```

9.1.2 定义时赋初值的 ROM (VHDL)

```

-- ROM Inference on array
library ieee;
use ieee.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
ENTITY gowin_rom is
    generic(
        Width : integer := 32;
        Depth : integer := 6
    );
    PORT (clk:IN STD_LOGIC;
          ce,rst:IN STD_LOGIC;

```

```

        addr : STD_LOGIC_VECTOR(Depth-1 DOWNT0 0);
        dout:OUT STD_LOGIC_VECTOR(Width-1 DOWNT0 0)
    );

end gowin_rom;

ARCHITECTURE rtl OF gowin_rom IS
    signal data_out :STD_LOGIC_VECTOR(Width-1 DOWNT0 0);
    TYPE vector_array IS ARRAY(2**Depth-1 DOWNT0 0) OF
    STD_LOGIC_VECTOR(Width-1 DOWNT0 0);
    CONSTANT mem:vector_array:=(
        X"52853fd5",X"38581bd2", X"040d53e4",
        X"22ce7d00",X"c0b4bf1c",X"ec45e626", X"d9d000d9", X"aacf8574",
        X"8c565693",X"b19808d0", X"e073036e",
        X"41b923f6",X"ba17fce1",X"d4dec5de", X"a18ad699", X"4a734008",
        X"8f26bdd4",X"b8d4aab6", X"f55e3c77",
        X"41a5d418",X"5c651d69",X"445469c3", X"2e49668b", X"dc1aa05b",
        X"1e1f0f1e",X"86fd31ef", X"73d90e02",
        X"b655bf16",X"dce89022",X"5c32ac0e", X"ba172648", X"cebfe4cd",
        X"32853fd5",X"3858abd2", X"04ac53e4",
        X"22ce7d00",X"c0b4bf1c",X"ec45e626", X"d9d000d9", X"aacf8574",
        X"8c565693",X"b19808d0", X"e073036e",
        X"41b923f6",X"ba17fce1",X"d4dec5de", X"a18ad699", X"4da34008",
        X"8f26bdd4",X"b8d4aab6", X"f55e3c77",
        X"41a5d418",X"5c651d69",X"445469c3", X"2e49668b", X"dc1aa05b",
        X"1e1f0f1e",X"86fd31ef", X"73d90e02",
        X"b655bf16",X"dce89022",X"5c32ac0e", X"ba172648", X"cebfe4cd"
    );
BEGIN
    PROCESS(clk)
    BEGIN
        IF(clk'EVENT AND clk='1') THEN
            IF(rst='1')THEN
                data_out<=(OTHERS=>'0');
            ELSE
                IF(ce='1') THEN
                    data_out<=mem(conv_integer(addr));
                END IF;
            END IF;
        END IF;
    END PROCESS;
END rtl;

```

```

        END IF;
    END IF;
END IF;
END PROCESS;
PROCESS(clk)
BEGIN
    IF(clk'EVENT AND clk='1') THEN
        IF(rst='1')THEN
            dout<=(OTHERS=>'0');
        ELSE
            IF(ce='1') THEN
                dout<=data_out;
            END IF;
        END IF;
    END IF;
END PROCESS;
END rtl;

```

9.1.3 Case 形式的 ROM (Verilog)

```

// ROMs Using Block RAM Resources
module gowin_rom(clk,rst,ce,addr,dout);
input clk;
input rst,ce;
input [5:0] addr;
output reg [31:0] dout;
reg [31:0] dout_reg;
always @(posedge clk )begin
    if (rst) begin
        dout_reg <= 0;
    end
    else begin
        if(ce)begin
            case(addr)
                6'h00: dout_reg <= 32'h52853fd5;
                6'h01: dout_reg <= 32'h38581bd2;

```

```
6'h02: dout_reg <= 32'h040d53e4;
6'h03: dout_reg <= 32'h22ce7d00;
6'h04: dout_reg <= 32'h73d90e02;
6'h05: dout_reg <= 32'hc0b4bf1c;
6'h06: dout_reg <= 32'hec45e626;
6'h07: dout_reg <= 32'hd9d000d9;
6'h08: dout_reg <= 32'haacf8574;
6'h09: dout_reg <= 32'hb655bf16;
6'h0a: dout_reg <= 32'h8c565693;
6'h0b: dout_reg <= 32'hb19808d0;
6'h0c: dout_reg <= 32'he073036e;
6'h0d: dout_reg <= 32'h41b923f6;
6'h0e: dout_reg <= 32'hdce89022;
6'h0f: dout_reg <= 32'hba17fce1;
6'h10: dout_reg <= 32'hd4dec5de;
6'h11: dout_reg <= 32'ha18ad699;
6'h12: dout_reg <= 32'h4a734008;
6'h13: dout_reg <= 32'h5c32ac0e;
6'h14: dout_reg <= 32'h8f26bdd4;
6'h15: dout_reg <= 32'hb8d4aab6;
6'h16: dout_reg <= 32'hf55e3c77;
6'h17: dout_reg <= 32'h41a5d418;
6'h18: dout_reg <= 32'hba172648;
6'h19: dout_reg <= 32'h5c651d69;
6'h1a: dout_reg <= 32'h445469c3;
6'h1b: dout_reg <= 32'h2e49668b;
6'h1c: dout_reg <= 32'hdc1aa05b;
6'h1d: dout_reg <= 32'hcebfe4cd;
6'h1e: dout_reg <= 32'h1e1f0f1e;
6'h1f: dout_reg <= 32'h86fd31ef;
6'h20: dout_reg <= 32'h05853fd5;
6'h21: dout_reg <= 32'h38581bd2;
6'h22: dout_reg <= 32'h040d53e4;
6'h23: dout_reg <= 32'h22ce7d00;
6'h24: dout_reg <= 32'h73d90e02;
```

```
        6'h25: dout_reg <= 32'hc0b4bf1c;
        6'h26: dout_reg <= 32'hec45e626;
        6'h27: dout_reg <= 32'hd9d000d9;
        6'h28: dout_reg <= 32'haacf8574;
        6'h29: dout_reg <= 32'hb655bf16;
        6'h2a: dout_reg <= 32'h8c565693;
        6'h2b: dout_reg <= 32'hb19808d0;
        6'h2c: dout_reg <= 32'he073036e;
        6'h2d: dout_reg <= 32'h41b923f6;
        6'h2e: dout_reg <= 32'hdce89022;
        6'h2f: dout_reg <= 32'hba17fce1;
        6'h30: dout_reg <= 32'hd4dec5de;
        6'h31: dout_reg <= 32'ha18ad699;
        6'h32: dout_reg <= 32'h4a734008;
        6'h33: dout_reg <= 32'h5c32ac0e;
        6'h34: dout_reg <= 32'h8f26bdd4;
        6'h35: dout_reg <= 32'hb8d4aab6;
        6'h36: dout_reg <= 32'hc55e3c77;
        6'h37: dout_reg <= 32'h41a5d418;
        6'h38: dout_reg <= 32'hba172648;
        6'h39: dout_reg <= 32'h5a651d69;
        6'h3a: dout_reg <= 32'h445469c3;
        6'h3b: dout_reg <= 32'h2e49668b;
        6'h3c: dout_reg <= 32'hdc1aa05b;
        6'h3d: dout_reg <= 32'hcebfe4cd;
        6'h3e: dout_reg <= 32'h1e1f0f1e;
        6'h3f: dout_reg <= 32'hf6fd31ef;
        default: dout_reg <= 32'h8e9008a6;
    endcase
end
end
end
always @(posedge clk )begin
    if (rst) begin
        dout <= 0;
```

```

        end
    else begin
        if(ce)begin
            dout<=dout_reg;
        end
    end
end
endmodule

```

9.1.4 Case 形式的 ROM (VHDL)

-- ROMs Using Block RAM Resources

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
ENTITY gowin_rom is
```

```
    PORT (
```

```
        clk:IN STD_LOGIC;
```

```
        ce,rst:IN STD_LOGIC;
```

```
        addr : STD_LOGIC_VECTOR(5 DOWNT0 0);
```

```
        dout :OUT STD_LOGIC_VECTOR(31 DOWNT0 0)
```

```
    );
```

```
end gowin_rom;
```

```
ARCHITECTURE rtl OF gowin_rom IS
```

```
    signal dout_reg : STD_LOGIC_VECTOR(31 DOWNT0 0);
```

```
BEGIN
```

```
    PROCESS(clk)
```

```
    BEGIN
```

```
        IF(clk'EVENT AND clk='1') THEN
```

```
            IF(rst='1')THEN
```

```
                dout<=(OTHERS=>'0');
```

```
            ELSE
```

```
                IF(ce='1')THEN
```

```
                    dout <= dout_reg;
```

```
                END IF;
```

```
        END IF;
    END IF;
END PROCESS;

PROCESS(clk)
BEGIN
    IF(clk'EVENT AND clk='1') THEN
        IF(rst='1')THEN
            dout_reg<=(OTHERS=>'0');
        ELSE
            IF(ce='1')THEN
                CASE addr IS
                    WHEN b"000000"=> dout_reg <= x"52853fd5";
                    WHEN b"000001"=> dout_reg <= x"38581bd2";
                    WHEN b"000010"=> dout_reg <= x"040d53e4";
                    WHEN b"000011"=> dout_reg <= x"22ce7d00";
                    WHEN b"000100"=> dout_reg <= x"73d90e02";
                    WHEN b"000101"=> dout_reg <= x"c0b4bf1c";
                    WHEN b"000110"=> dout_reg <= x"ec45e626";
                    WHEN b"000111"=> dout_reg <= x"d9d000d9";
                    WHEN b"001000"=> dout_reg <= x"aacf8574";
                    WHEN b"001001"=> dout_reg <= x"b655bf16";
                    WHEN b"001010"=> dout_reg <= x"8c565693";
                    WHEN b"001011"=> dout_reg <= x"b19808d0";
                    WHEN b"001100"=> dout_reg <= x"e073036e";
                    WHEN b"001101"=> dout_reg <= x"41b923f6";
                    WHEN b"001110"=> dout_reg <= x"dce89022";
                    WHEN b"001111"=> dout_reg <= x"ba17fce1";
                    WHEN b"010000"=> dout_reg <= x"d4dec5de";
                    WHEN b"010001"=> dout_reg <= x"a18ad699";
                    WHEN b"010010"=> dout_reg <= x"4a734008";
                    WHEN b"010011"=> dout_reg <= x"5c32ac0e";
                    WHEN b"010100"=> dout_reg <= x"8f26bdd4";
                    WHEN b"010101"=> dout_reg <= x"b8d4aab6";
                    WHEN b"010110"=> dout_reg <= x"f55e3c77";
```

```
WHEN b"010111"=> dout_reg <= x"41a5d418";
WHEN b"011000"=> dout_reg <= x"ba172648";
WHEN b"011001"=> dout_reg <= x"5c651d69";
WHEN b"011010"=> dout_reg <= x"445469c3";
WHEN b"011011"=> dout_reg <= x"2e49668b";
WHEN b"011100"=> dout_reg <= x"dc1aa05b";
WHEN b"011101"=> dout_reg <= x"cebfe4cd";
WHEN b"011110"=> dout_reg <= x"1e1f0f1e";
WHEN b"011111"=> dout_reg <= x"86fd31ef";
WHEN b"100000"=> dout_reg <= x"fa853fd5";
WHEN b"100001"=> dout_reg <= x"38581bd2";
WHEN b"100010"=> dout_reg <= x"f40d53e4";
WHEN b"100011"=> dout_reg <= x"22ce7d00";
WHEN b"100100"=> dout_reg <= x"73d90e02";
WHEN b"100101"=> dout_reg <= x"c0b4bf1c";
WHEN b"100110"=> dout_reg <= x"ec45e626";
WHEN b"100111"=> dout_reg <= x"d9d000d9";
WHEN b"101000"=> dout_reg <= x"aacf8574";
WHEN b"101001"=> dout_reg <= x"b655bf16";
WHEN b"101010"=> dout_reg <= x"8c565693";
WHEN b"101011"=> dout_reg <= x"b19808d0";
WHEN b"101100"=> dout_reg <= x"e073036e";
WHEN b"101101"=> dout_reg <= x"41b923f6";
WHEN b"101110"=> dout_reg <= x"dce89022";
WHEN b"101111"=> dout_reg <= x"ba17fce1";
WHEN b"110000"=> dout_reg <= x"d4dec5de";
WHEN b"110001"=> dout_reg <= x"a18ad699";
WHEN b"110010"=> dout_reg <= x"fa734008";
WHEN b"110011"=> dout_reg <= x"cc32ac0e";
WHEN b"110100"=> dout_reg <= x"af26bdd4";
WHEN b"110101"=> dout_reg <= x"b8d4aab6";
WHEN b"110110"=> dout_reg <= x"f55e3c77";
WHEN b"110111"=> dout_reg <= x"41a5d418";
WHEN b"111000"=> dout_reg <= x"ba172648";
WHEN b"111001"=> dout_reg <= x"5c651d69";
```

```

        WHEN b"111010"=> dout_reg <= x"445469c3";
        WHEN b"111011"=> dout_reg <= x"2e49668b";
        WHEN b"111100"=> dout_reg <= x"dc1aa05b";
        WHEN b"111101"=> dout_reg <= x"cebfe4cd";
        WHEN b"111110"=> dout_reg <= x"1e1f0f1e";
        WHEN b"111111"=> dout_reg <= x"86fd31ef";
        WHEN OTHERS => dout_reg <= x"8e9008a6";
        END CASE;
    END IF;
END IF;
END IF;
END PROCESS;
END rtl;

```

9.1.5 initial 块赋初值的 ROM (Verilog)

```

// ROM Inference on array
module gowin_rom(dout, addr,clk, ce,rst);
output reg[9:0]dout;
input [9:0]addr;
input clk, ce,rst;
reg [9:0] mem [1023:0];
integer i;
initial begin
    for (i=0; i<1023; i=i+1)
        mem[i] = i;
end
always @(posedge clk)begin
    if (rst)begin
        dout<=0;
    end
    else begin
        if(ce)begin
            dout<= mem[addr];
        end
    end
end
end

```

```
end
endmodule
```

9.1.6 readmemh 赋初值的 ROM (Verilog)

```
// ROM Inference on array
module gowin_rom(dout,addr, clk, ce,rst);
output reg[31:0]dout;
input [5:0]addr;
input clk, ce,rst;
reg [31:0] mem [63:0];
```

```
initial begin
    $readmemh("E:/ram.mi", mem);
end
```

```
always @(posedge clk)begin
    if (rst)begin
        dout<=0;
    end
    else begin
        if(ce)begin
            dout<= mem[addr];
        end
    end
end
```

```
end
endmodule
```

ram.mi 数据书写形式如下:

```
52853fd5
38581bd2
040d53e4
22ce7d00
c0b4bf1c
ec45e626
d9d000d9
aacf8574
```

8c565693
b19808d0
e073036e
41b923f6
ba17fce1
d4dec5de
a18ad699
4a734008
8f26bdd4
b8d4aab6
f55e3c77
41a5d418
5c651d69
445469c3
2e49668b
dc1aa05b
1e1f0f1e
86fd31ef
73d90e02
b655bf16
dce89022
5c32ac0e
ba172648
cebfe4cd
32853fd5
3858abd2
04ac53e4
22ce7d00
c0b4bf1c
ec45e626
d9d000d9
aacf8574
8c565693
b19808d0
e073036e

```
41b923f6
ba17fce1
d4dec5de
a18ad699
4da34008
8f26bdd4
b8d4aab6
f55e3c77
41a5d418
5c651d69
445469c3
2e49668b
dc1aa05b
1e1f0f1e
86fd31ef
73d90e02
b655bf16
dce89022
5c32ac0e
ba172648
cebfe4cd
```

9.1.7 readmemb 赋初值的 ROM (Verilog)

```
// ROM Inference on array
module gowin_rom(dout,addr, clk, ce,rst);
output reg[31:0]dout;
input [5:0]addr;
input clk, ce,rst;
reg [31:0] mem [63:0];

initial begin
    $readmemb("E:/ram.mi", mem);
end

always @(posedge clk)begin
```

```

        if (rst)begin
            dout<=0;
        end
        else begin
            if(ce)begin
                dout<= mem[addr];
            end
        end
    end
end
endmodule

```

ram.mi 数据书写形式如下:

```

00000001001000111111011101110111
01000101101100110000000100100011
10001001101010111101011010001011
11010110100010111110110110110011
10000001010101011101011010001011
11110111011101111100111011011001
11100101101100111100111011011001
10011000111000001000100110101011
11000001001000111110011101110111
11000101101100111110010110110011
10101001101010110110100110101011
10010110100010111110011101110111
10100001010101011001100011100000
11100111011101111010000101010101
11101101101100110110100110101011
10010000111000001110011101110111
11000010010001101100000100100011
01000101101100101000100110101011
01101001101010111110010110110011
11010110100010111001100011100000
01100001010101011110011101110111
11001110110110010110000101010101
10011000111000001111011101100111

```

```
11000001001000110000000101010101
11000101101100111000100110101011
1010100110101011101101110110111
10010110100010111111000011111111
10100001010101011111000011011011
11100111011011111111000011110011
11101101101100111111000011111111
10010000111111111110010110100011
00000001001000111111000011111111
00000001001000111111011101110111
01000101101100110000000100100011
10001001101010111101011010001011
11010110100010111110110110110011
10000001010101011101011010001011
11110111011101111100111011011001
11100101101100111100111011011001
10011000111000001000100110101011
11000001001000111110011101110111
11000101101100111110010110110011
10101001101010110110100110101011
10010110100010111110011101110111
10100001010101011001100011100000
11100111011101111010000101010101
11101101101100110110100110101011
10010000111000001110011101110111
11000010010001101100000100100011
01000101101100101000100110101011
01101001101010111110010110110011
11010110100010111001100011100000
01100001010101011110011101110111
11001110110110010110000101010101
10011000111000001111011101100111
11000001001000110000000101010101
11000101101100111000100110101011
10101001101010111011011101110111
```

```

10010110100010111111000011111111
10100001010101011111000011011011
11100111011101111111000011110011
11101101101100111111000011111111
10010000111111111110010110100011
00000001001000111111000011111111

```

9.1.8 readline 赋初值的 ROM (VHDL)

```

-- ROM Inference on array
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use std.textio.all;
entity gowin_rom is
    port(
        clk : in std_logic;
        ce : in std_logic;
        addr : in std_logic_vector(5 downto 0);
        dout : out std_logic_vector(31 downto 0)
    );
end gowin_rom;
architecture syn of gowin_rom is
    type RamType is array (0 to 63) of bit_vector(31 downto 0);
    impure function InitRam(RamFileName : in string) return
RamType is
        FILE RamFile : text is in RamFileName;
        variable RamFileLine : line;
        variable RAM : RamType;
        begin
            for i in RamType'range loop
                readline(RamFile, RamFileLine);
                read(RamFileLine, RAM(i));
            end loop;
            return RAM;
        end function;
    signal RAM : RamType := InitRam("E:/ram.mi");

```

```

begin
process(clk)
begin
    if clk'event and clk = '1' then
        if ce = '1' then
            dout <=
to_stdlogicvector(RAM(to_integer(unsigned(addr))));
        end if;
    end if;
end process;
end syn;

```

ram.mi 数据书写形式如下:

```

00000001001000111111011101110111
01000101101100110000000100100011
10001001101010111101011010001011
11010110100010111110110110110011
10000001010101011101011010001011
11110111011101111100111011011001
11100101101100111100111011011001
10011000111000001000100110101011
11000001001000111111001110111011
11000101101100111111001011011001
10101001101010110110100110101011
10010110100010111111001110111011
10100001010101011001100011100000
11100111011101111101000010101010
11101101101100110110100110101011
10010000111000001110011101110111
11000010010001101100000100100011
01000101101100101000100110101011
01101001101010111111001011011001
11010110100010111100110001110000
01100001010101011111001110111011
11001110110110010110000101010101
10011000111000001111011101100111

```

```
11000001001000110000000101010101
11000101101100111000100110101011
10101001101010111011011101110111
10010110100010111111000011111111
10100001010101011111000011011011
11100111011011111111000011110011
11101101101100111111000011111111
10010000111111111110010110100011
00000001001000111111000011111111
00000001001000111111011101110111
01000101101100110000000100100011
10001001101010111101011010001011
11010110100010111110110110110011
10000001010101011101011010001011
11110111011101111100111011011001
11100101101100111100111011011001
10011000111000001000100110101011
11000001001000111110011101110111
11000101101100111110010110110011
10101001101010110110100110101011
10010110100010111110011101110111
10100001010101011001100011100000
11100111011101111010000101010101
11101101101100110110100110101011
10010000111000001110011101110111
11000010010001101100000100100011
01000101101100101000100110101011
01101001101010111110010110110011
11010110100010111001100011100000
01100001010101011110011101110111
11001110110110010110000101010101
10011000111000001111011101100111
11000001001000110000000101010101
11000101101100111000100110101011
10101001101010111011011101110111
```

```
10010110100010111111000011111111
10100001010101011111000011011011
11100111011101111111000011110011
11101101101100111111000011111111
1001000011111111110010110100011
00000001001000111111000011111111
```

10_{DSP}

10.1 乘法器

10.1.1 乘法器示例

无符号数 9X9 位的乘法 (Verilog)

```
// Unsigned 9X9 bit Multiplier
// 1 latency stage on operands
// 2 latency stage after the multiplication
module gowin_dsp(a,b,clk,ce,rst,out);
input [8:0]a;
input [8:0]b;
input clk,ce,rst;
output [17:0]out;
reg [8:0]a_reg;
reg [8:0]b_reg;
reg [17:0]pp_reg,out_reg;
always@(posedge clk or posedge rst)begin
    if (rst)begin
        a_reg<=0;
        b_reg<=0;
    end
    else begin
        if(ce)begin
            a_reg<=a;
            b_reg<=b;
        end
    end
end
```

```

        end
    end
    always@(posedge clk or posedge rst)begin
        if (rst)begin
            pp_reg<=0;
        end
        else begin
            if(ce)begin
                pp_reg<=a_reg*b_reg;
            end
        end
    end
    always@(posedge clk or posedge rst)begin
        if (rst)begin
            out_reg<=0;
        end
        else begin
            if(ce)begin
                out_reg<=pp_reg;
            end
        end
    end
    assign out=out_reg;
endmodule

```

无符号数 9X9 位的乘法 (VHDL)

```

-- Unsigned 9x9 bit Multiplier
-- 1 latency stage on operands
-- 2 latency stage after the multiplication
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity gowin_dsp is
    generic(

```

```

        a_width : integer := 9;
        b_width : integer := 9;
        dout_width : integer := 18
    );
    port (
        dout: out unsigned (dout_width-1 downto 0);
        a : in unsigned (a_width-1 downto 0);
        b : in unsigned (b_width-1 downto 0);
        clk,ce,rst:in std_logic
    );
end gowin_dsp;
architecture Behavioral of gowin_dsp is
    signal a_reg : unsigned (a_width-1 downto 0);
    signal b_reg : unsigned (b_width-1 downto 0);
    signal pp_reg,out_reg:unsigned (dout_width-1 downto 0);
begin
    process(clk,rst)begin
        if(rst='1')then
            a_reg<=(others=>'0');
            b_reg<=(others=>'0');
        else
            if(rising_edge(clk))then
                if(ce='1')then
                    a_reg<=a;
                    b_reg<=b;
                end if;
            end if;
        end if;
    end process;
    process(clk,rst)begin
        if(rst='1')then
            pp_reg<=(others=>'0');
        else
            if(rising_edge(clk))then
                if(ce='1')then

```

```

        pp_reg<=a_reg*b_reg;
    end if;
    end if;
    end if;
end process;
process(clk,rst)begin
    if(rst='1')then
        out_reg<=(others=>'0');
    else
        if(rising_edge(clk))then
            if(ce='1')then
                out_reg<=pp_reg;
            end if;
        end if;
    end if;
end process;
dout<=out_reg;
end Behavioral;

```

有符号数 18X18 位的乘法（Verilog）

```

// Signed 18x18 bit Multiplier
// 1 latency stage on operands
// 2 latency stage after the multiplication
module gowin_dsp(a,b,clk,ce,rst,out);
    input signed[17:0]a;
    input signed[17:0]b;
    input clk,ce,rst;
    output signed[35:0]out;
    reg signed[17:0]a_reg;
    reg signed[17:0]b_reg;
    reg signed[35:0]pp_reg,out_reg;
    always@(posedge clk)begin
        if (rst)begin
            a_reg<=0;
            b_reg<=0;
        end
    end
end

```

```

        else begin
            if(ce)begin
                a_reg<=a;
                b_reg<=b;
            end
        end
    end
end
always@(posedge clk)begin
    if (rst)begin
        pp_reg<=0;
    end
    else begin
        if(ce)begin
            pp_reg<=a_reg*b_reg;
        end
    end
end
always@(posedge clk)begin
    if (rst)begin
        out_reg<=0;
    end
    else begin
        if(ce)begin
            out_reg<=pp_reg;
        end
    end
end
end

assign out=out_reg;
endmodule

```

有符号数 18X18 位的乘法 (VHDL)

```

-- Signed 18x18 bit Multiplier
-- 1 latency stage on operands
-- 2 latency stage after the multiplication
library IEEE;

```

```

use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity gowin_dsp is
    generic(
        a_width : integer := 18;
        b_width : integer := 18;
        dout_width : integer := 36
    );
    port (
        dout: out signed (dout_width-1 downto 0);
        a : in signed (a_width-1 downto 0);
        b : in signed (b_width-1 downto 0);
        clk,ce,rst:in std_logic
    );
end gowin_dsp;

architecture Behavioral of gowin_dsp is
    signal a_reg : signed (a_width-1 downto 0);
    signal b_reg : signed (b_width-1 downto 0);
    signal pp_reg,out_reg: signed (dout_width-1 downto 0);
begin
    process(clk,rst)begin
        if(rising_edge(clk))then
            if(rst='1')then
                a_reg<=(others=>'0');
                b_reg<=(others=>'0');
            else
                if(ce='1')then
                    a_reg<=a;
                    b_reg<=b;
                end if;
            end if;
        end if;
    end process;
end process;

```

```

process(clk,rst)begin
    if(rising_edge(clk))then
        if(rst='1')then
            pp_reg<=(others=>'0');
        else
            if(ce='1')then
                pp_reg<=a_reg*b_reg;
            end if;
        end if;
    end if;
end process;

```

```

process(clk,rst)begin
    if(rising_edge(clk))then
        if(rst='1')then
            out_reg<=(others=>'0');
        else
            if(ce='1')then
                out_reg<=pp_reg;
            end if;
        end if;
    end if;
end process;
dout<=out_reg;
end Behavioral;

```

10.2 乘加器

10.2.1 静态乘加/减示例

乘法器的静态乘加（Verilog）

```

// Static multiply-accumulate of unsigned 12x12 bit multiplier
// 1 latency stage on operands
// 2 latency stage after the multiplication
module gowin_dsp (clk,ce,rst,a,b,d0,d1,dout);
    input clk,ce,rst;
    input [11:0]a;

```

```
input [11:0]b;
input [11:0]d0;
input [11:0]d1;
output reg [24:0]dout;

reg [11:0]a_reg;
reg [11:0]b_reg;
reg [11:0]d0_reg;
reg [11:0]d1_reg;
reg [23:0]preg0,preg1;
always@(posedge clk)begin
    if(rst)begin
        a_reg<=0;
        b_reg<=0;
        d0_reg<=0;
        d1_reg<=0;
    end
    else begin
        if(ce)begin
            a_reg<=a;
            b_reg<=b;
            d0_reg<=d0;
            d1_reg<=d1;
        end
    end
end

always@(posedge clk)begin
    if(rst)begin
        dout<=0;
        preg0<=0;
        preg1<=0;
    end
    else begin
        if(ce)begin
```

```

        dout<=preg0+preg1;
        preg0<=a_reg*b_reg;
        preg1<=d0_reg*d1_reg;
    end
end
end
endmodule

```

乘法器的静态乘减 (VHDL)

```

-- Static multiplication and subtraction of 2 multipliers
-- 1 latency stage on operands
-- 2 latency stage after the multiplication
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity gowin_dsp is
    generic(
        a_width : integer := 26;
        b_width : integer := 17;
        dout_width : integer := 43
    );
    port (
        dout: out unsigned (dout_width-1 downto 0);
        a,d0 : in unsigned (a_width-1 downto 0);
        b,d1 : in unsigned (b_width-1 downto 0);
        clk,ce,rst: in std_logic
    );
end gowin_dsp;

architecture Behavioral of gowin_dsp is
    signal mult1,mult2 : unsigned (dout_width-1 downto 0);
    signal a_reg,d0_reg : unsigned (a_width-1 downto 0);
    signal b_reg,d1_reg : unsigned (b_width-1 downto 0);
begin
    process(clk)begin
        if(rising_edge(clk))then

```

```

        if(rst='1')then
            a_reg<=(others=>'0');
            b_reg<=(others=>'0');
            d0_reg<=(others=>'0');
            d1_reg<=(others=>'0');
        else
            if(ce='1')then
                a_reg<=a;
                b_reg<=b;
                d0_reg<=d0;
                d1_reg<=d1;
            end if;
        end if;
    end if;
end process;

process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            mult1<=(others=>'0');
            mult2<=(others=>'0');
            dout<=(others=>'0');
        else
            if(ce='1')then
                mult1<=a_reg*b_reg;
                mult2<=d0_reg*d1_reg;
                dout<=mult1-mult2;
            end if;
        end if;
    end if;
end process;
end Behavioral;

```

乘法器与加数的静态乘加（Verilog）

```

// Static multiplication and addition of multipliers and addends
// 1 latency stage on operands

```

```
// 2 latency stage after the multiplication
module gowin_dsp(a,b,c,clk,ce,rst,out);
input [11:0]a;
input [11:0]b;
input [46:0]c;
input  clk,ce,rst;
output [46:0]out;
reg [11:0]a_reg;
reg [11:0]b_reg;
reg [46:0]c_reg;

reg [46:0]pp_reg,out_reg;

always@(posedge clk)begin
    if (rst)begin
        a_reg<=0;
        b_reg<=0;
        c_reg<=0;
    end
    else begin
        if(ce)begin
            a_reg<=a;
            b_reg<=b;
            c_reg<=c;
        end
    end
end

always@(posedge clk)begin
    if (rst)begin
        pp_reg<=0;
    end
    else begin
        if(ce)begin
            pp_reg<=a_reg*b_reg;
        end
    end
end
```

```

        end
    end
    always@(posedge clk)begin
        if (rst)begin
            out_reg<=0;
        end
        else begin
            if(ce)begin
                out_reg<= pp_reg + c_reg ;
            end
        end
    end
end

assign out=out_reg;
endmodule

```

乘法器与加数的静态乘减 (VHDL)

```

-- Static multiplication and subtraction of multipliers and addends
-- 1 latency stage on operands
-- 2 latency stage after the multiplication
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;

entity gowin_dsp is
    generic(
        a_width : integer := 12;
        b_width : integer := 12;
        c_width : integer := 47;
        dout_width : integer := 47
    );
    port (
        dout: out std_logic_vector (dout_width-1 downto 0);
        a : in std_logic_vector (a_width-1 downto 0);
        b : in std_logic_vector (b_width-1 downto 0);
        c : in std_logic_vector (c_width-1 downto 0);

```

```

        clk,ce,rst:in std_logic
    );
end gowin_dsp;
architecture Behavioral of gowin_dsp is
    signal a_reg : std_logic_vector (a_width-1 downto 0);
    signal b_reg : std_logic_vector (b_width-1 downto 0);
    signal c_reg : std_logic_vector (c_width-1 downto 0);
    signal pp_reg: std_logic_vector (a_width+b_width-1 downto 0);
    signal out_reg: std_logic_vector (dout_width-1 downto 0);
begin

    process(clk)begin
        if(rising_edge(clk))then
            if(rst='1')then
                a_reg<=(others=>'0');
                b_reg<=(others=>'0');
                c_reg<=(others=>'0');
            else
                if(ce='1')then
                    a_reg<=a;
                    b_reg<=b;
                    c_reg<=c;
                end if;
            end if;
        end if;
    end process;
    process(clk)begin
        if(rising_edge(clk))then
            if(rst='1')then
                pp_reg<=(others=>'0');
            else
                if(ce='1')then
                    pp_reg<=a_reg*b_reg;
                end if;
            end if;
        end if;
    end process;
end architecture Behavioral of gowin_dsp;

```

```

        end if;
    end process;

    process(clk)begin
        if(rising_edge(clk))then
            if(rst='1')then
                out_reg<=(others=>'0');
            else
                if(ce='1')then
                    out_reg<= pp_reg - c_reg;
                end if;
            end if;
        end if;
    end process;
    dout<=out_reg;
end Behavioral;

```

10.2.2 动态乘加/减示例

乘法器的动态乘加/减（Verilog）

```

//Dynamic multiply-add/subtract with 2-bit multiplier
// 1 latency stage on operands
// 2 latency stage after the multiplication
module gowin_dsp(a0,a1,b0,b1,clk,ce,rst,addsub,dout);
    input signed[11:0]a0,a1;
    input signed[11:0]b0,b1;
    input clk,ce,rst;
    input [1:0] addsub;
    output signed[23:0]dout;
    reg signed[11:0]a0_reg,a1_reg;
    reg signed[11:0]b0_reg,b1_reg;

    reg signed[23:0]p0_reg,p1_reg,dout_reg;

    always@(posedge clk)begin
        if (rst)begin

```

```

        a0_reg<=0;
        b0_reg<=0;
        a1_reg<=0;
        b1_reg<=0;
    end
    else begin
        if(ce)begin
            a0_reg<=a0;
            b0_reg<=b0;
            a1_reg<=a1;
            b1_reg<=b1;
        end
    end
end
always@(posedge clk)begin
    if (rst)begin
        p0_reg<=0;
        p1_reg<=0;
    end
    else begin
        if(ce)begin
            p0_reg<=a0_reg*b0_reg;
            p1_reg<=a1_reg*b1_reg;
        end
    end
end
always@(posedge clk)begin
    if (rst)begin
        dout_reg<=0;
    end
    else begin
        if(ce)begin
            dout_reg<= addsub==2'b00 ? p0_reg + p1_reg :
                        addsub==2'b01 ? p1_reg -p0_reg  :
                        addsub==2'b10 ? p0_reg - p1_reg : -p0_reg-

```

```

p1_reg;
        end
    end
end

assign dout=dout_reg;
endmodule

```

乘法器的动态乘加/减 (VHDL)

```

-- Multiply-add/subtract select with Dynamic control
-- 1 latency stage on operands
-- 2 latency stage after the multiplication
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity gowin_dsp is
    generic(
        a_width : integer := 11;
        b_width : integer := 11;
        dout_width : integer := 22
    );
    port (
        dout: out unsigned (dout_width-1 downto 0);
        a0,a1 : in unsigned (a_width-1 downto 0);
        b0,b1 : in unsigned (b_width-1 downto 0);
        addsub : in std_logic_vector(1 downto 0);
        clk,ce,rst:in std_logic
    );
end gowin_dsp;

architecture Behavioral of gowin_dsp is
    signal a0_reg,a1_reg : unsigned (a_width-1 downto 0);
    signal b0_reg,b1_reg : unsigned (b_width-1 downto 0);
    signal p0_reg,p1_reg :unsigned (a_width+b_width-1 downto 0);
    signal dout_reg :unsigned (dout_width-1 downto 0);
begin
    process(clk)begin

```

```
    if(rising_edge(clk))then
        if(rst='1')then
            a0_reg<=(others=>'0');
            b0_reg<=(others=>'0');
            a1_reg<=(others=>'0');
            b1_reg<=(others=>'0');
        else
            if(ce='1')then
                a0_reg<=a0;
                b0_reg<=b0;
                a1_reg<=a1;
                b1_reg<=b1;
            end if;
        end if;
    end if;
end process;
process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            p0_reg<=(others=>'0');
            p1_reg<=(others=>'0');
        else
            if(ce='1')then
                p0_reg<=a0_reg*b0_reg;
                p1_reg<=a1_reg*b1_reg;
            end if;
        end if;
    end if;
end process;

process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            dout_reg<=(others=>'0');
        else
            dout_reg<=p0_reg+p1_reg;
        end if;
    end if;
end process;
```

```

        if(ce='1')then
            if(addsub="00")then
                dout_reg <= p0_reg + p1_reg;
            elsif(addsub="01")then
                dout_reg <= '0'-p0_reg+p1_reg ;
            elsif(addsub="10")then
                dout_reg <= p0_reg - p1_reg;
            else
                dout_reg <= '0'-p0_reg - p1_reg;
            end if;
        end if;
    end if;
end process;
dout<=dout_reg;
end Behavioral;

```

乘法器与加数的动态乘加/减（Verilog）

```

// Multiply-add/subtract select with Dynamic control
// 1 latency stage on operands
// 2 latency stage after the multiplication
module gowin_dsp (a,b,c,clk,ce,rst,sel,addsub,out);
input signed[11:0]a;
input signed[11:0]b;
input signed[47:0]c;
input clk,ce,rst;
input sel;
input[1:0] addsub;
output signed[47:0]out;
reg signed[11:0]a_reg;
reg signed[11:0]b_reg;
reg signed[47:0]c_reg;

reg signed[47:0]pp_reg,out_reg;
wire signed[47:0]c_sig;
assign c_sig=sel ? c_reg : 0;

```

```

always@(posedge clk)begin
    if (rst)begin
        a_reg<=0;
        b_reg<=0;
        c_reg<=0;
    end
    else begin
        if(ce)begin
            a_reg<=a;
            b_reg<=b;
            c_reg<=c;
        end
    end
end

always@(posedge clk)begin
    if (rst)begin
        pp_reg<=0;
    end
    else begin
        if(ce)begin
            pp_reg<=a_reg*b_reg;
        end
    end
end

always@(posedge clk)begin
    if (rst)begin
        out_reg<=0;
    end
    else begin
        if(ce)begin
            out_reg<= addsub==2'b00 ? pp_reg + c_sig :
                        addsub==2'b01 ? -pp_reg + c_sig:
                        addsub==2'b10 ? pp_reg - c_sig : -pp_reg-
c_sig;
        end
    end
end

```

```

        end
    end

    assign out=out_reg;
endmodule

```

乘法器与加数的动态乘加/减 (VHDL)

```

-- Multiply-add/subtract select with Dynamic control
-- 1 latency stage on operands
-- 2 latency stage after the multiplication
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity gowin_dsp is
    generic(
        a_width : integer := 12;
        b_width : integer := 12;
        c_width : integer := 48;
        dout_width : integer := 48
    );
    port (
        dout: out signed (dout_width-1 downto 0);
        a : in signed (a_width-1 downto 0);
        b : in signed (b_width-1 downto 0);
        c : in signed (c_width-1 downto 0);
        sel : in std_logic;
        addsub : in std_logic_vector(1 downto 0);
        clk,ce,rst:in std_logic
    );
end gowin_dsp;
architecture Behavioral of gowin_dsp is
    signal a_reg : signed (a_width-1 downto 0);
    signal b_reg : signed (b_width-1 downto 0);
    signal c_reg : signed (c_width-1 downto 0);
    signal pp_reg: signed (a_width+b_width-1 downto 0);
    signal c_sig: signed (c_width-1 downto 0);

```

```
    signal out_reg : signed (dout_width-1 downto 0);
begin
    process(sel,c_reg)begin
        if(sel='1')then
            c_sig<=c_reg;
        else
            c_sig<=(others=>'0');
        end if;
    end process;

    process(clk)begin
        if(rising_edge(clk))then
            if(rst='1')then
                a_reg<=(others=>'0');
                b_reg<=(others=>'0');
                c_reg<=(others=>'0');
            else
                if(ce='1')then
                    a_reg<=a;
                    b_reg<=b;
                    c_reg<=c;
                end if;
            end if;
        end if;
    end process;

    process(clk)begin
        if(rising_edge(clk))then
            if(rst='1')then
                pp_reg<=(others=>'0');
            else
                if(ce='1')then
                    pp_reg<=a_reg*b_reg;
                end if;
            end if;
        end if;
    end process;
```

```

end process;

process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            out_reg<=(others=>'0');
        else
            if(ce='1')then
                if(addsub="00")then
                    out_reg<=pp_reg + c_sig;
                elsif(addsub="01")then
                    out_reg<= -pp_reg+c_sig ;
                elsif(addsub="10")then
                    out_reg<= pp_reg - c_sig;
                else
                    out_reg<= -pp_reg - c_sig;
                end if;
            end if;
        end if;
    end if;
end process;

dout<=out_reg;
end Behavioral;

```

10.3 累加器

10.3.1 静态累加示例

```

 $\sum(A*B)$  (Verilog)
// Static accumulation of multiplier( $\sum(A*B)$ )
// 1 latency stage on operands
// 2 latency stage after the multiplication
module gowin_dsp (a,b,clk,ce,rst,out);
parameter widtha=26;
parameter widthb=17;

```

```
input [widtha-1:0]a;
input [widthb-1:0]b;
input clk,ce,rst;
output [widtha+widthb-1:0]out;
reg [widtha+widthb-1:0]out_reg;
reg [widtha-1:0]a_reg;
reg [widthb-1:0]b_reg;
wire [widtha+widthb-1:0]s_sel;
reg [widtha+widthb-1:0]mout;
always@(posedge clk)begin
    if (rst)begin
        a_reg<=0;
        b_reg<=0;
    end
    else begin
        if(ce)begin
            a_reg<=a;
            b_reg<=b;
        end
    end
end
assign s_sel= out_reg;

always@(posedge clk)begin
    if (rst)begin
        out_reg<=0;
        mout<=0;
    end
    else begin
        if(ce)begin
            mout<=a_reg*b_reg;
            out_reg<=s_sel+mout;
        end
    end
end
end
```

```

    assign out=out_reg;
endmodule

```

$\Sigma(A*B)$ (VHDL)

```

-- Static accumulation of unsigned 26x17 bit multiplier ( $\Sigma(A*B)$ )
-- 1 latency stage on operands
-- 2 latency stage after the multiplication

```

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

```

```

entity gowin_dsp is

```

```

    generic(
        a_width : integer := 26;
        b_width : integer := 17;
        dout_width : integer := 43
    );

```

```

    port (
        dout: out signed (dout_width-1 downto 0);
        a : in signed (a_width-1 downto 0);
        b : in signed (b_width-1 downto 0);
        clk,ce,rst:in std_logic
    );

```

```

end gowin_dsp;

```

```

architecture Behavioral of gowin_dsp is

```

```

    signal mout :signed (dout_width -1 downto 0);
    signal s_sel :signed (dout_width -1 downto 0);
    signal out_reg :signed (dout_width-1 downto 0);
    signal a_reg : signed (a_width-1 downto 0);
    signal b_reg : signed (b_width-1 downto 0);

```

```

begin

```

```

    process(clk)begin
        if(rising_edge(clk))then
            if(rst='1')then
                a_reg<=(others=>'0');

```

```

        b_reg<=(others=>'0');
    else
        if(ce='1')then
            a_reg<=a;
            b_reg<=b;
        end if;
    end if;
end if;
end process;

s_sel<=out_reg;
process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            out_reg<=(others=>'0');
            mout<=(others=>'0');
        else
            if(ce='1')then
                mout<=a_reg*b_reg;
                out_reg<=s_sel+mout;
            end if;
        end if;
    end if;
end process;
dout<=out_reg;
end Behavioral;

```

$\Sigma(A0*B0 \pm A1*B1)$ (Verilog)

```

// Static accumulation after multiplier summation( $\Sigma(A0*B0+A1*B1)$ )
// 1 latency stage on operands
// 2 latency stage after the multiplication
module gowin_dsp(a0, a1, b0, b1, s, accload, ce, reset, clk);
parameter a_width=9;
parameter b_width=9;
parameter s_width=18;
input unsigned [a_width-1:0] a0, a1;

```

```
input unsigned [b_width-1:0] b0, b1;
input accload, ce, reset, clk;
output unsigned [s_width-1:0] s;
wire unsigned [s_width-1:0] s_sel;
wire unsigned [s_width-1:0] p0, p1;
reg unsigned [s_width-1:0] p0_reg=0;
reg unsigned [s_width-1:0] p1_reg=0;
reg unsigned [s_width-1:0] s=0;
reg acc_reg0=1'b0;
reg acc_reg1=1'b0;
reg unsigned [a_width-1:0] a0_reg, a1_reg;
reg unsigned [b_width-1:0] b0_reg, b1_reg;

always @(posedge clk or posedge reset)begin
    if(reset) begin
        a0_reg <= 0;
        a1_reg <= 0;
        b0_reg <= 0;
        b1_reg <= 0;
    end else if(ce) begin
        a0_reg <= a0;
        a1_reg <= a1;
        b0_reg <= b0;
        b1_reg <= b1;
    end
end

assign p0 = a0_reg*b0_reg;
assign p1 = a1_reg*b1_reg;
always @(posedge clk or posedge reset)begin
    if(reset) begin
        p0_reg <= 0;
        p1_reg <= 0;
    end else if(ce) begin
        p0_reg <= p0;
    end
end
```

```

        p1_reg <= p1;
    end
end
always @(posedge clk)begin
    acc_reg0 <= accload;
    acc_reg1 <= acc_reg0;
end
assign s_sel = (acc_reg0 == 1) ? s : 0;
always @(posedge clk or posedged reset)begin
    if(reset) begin
        s <= 0;
    end else if(ce) begin
        s <= s_sel + p0_reg + p1_reg;
    end
end
endmodule

```

$\Sigma(A0*B0 \pm A1*B1)$ (VHDL)

```

-- Static accumulation after multiplier summation ( $\Sigma(A0*B0-A1*B1)$ )
-- 1 latency stage on operands
-- 2 latency stage after the multiplication

```

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

```

```

entity gowin_dsp is
    generic(
        a0_width : integer := 12;
        a1_width : integer := 12;
        b0_width : integer := 12;
        b1_width : integer := 12;
        s_width : integer := 24
    );
    port (
        s: buffer signed(s_width-1 downto 0);
        a0 : in signed(a0_width-1 downto 0);

```

```

        a1 : in signed(a0_width-1 downto 0);
        b0 : in signed(b0_width-1 downto 0);
        b1 : in signed(b0_width-1 downto 0);
        accload: in std_logic;
        ce: in std_logic;
        clock: in std_logic;
        reset: in std_logic
    );
end gowin_dsp;
architecture Behavioral of gowin_dsp is
    signal p0: signed(a0_width+b0_width-1 downto 0);
    signal p1: signed(a1_width+b1_width-1 downto 0);
    signal p0_reg: signed(s_width-1 downto 0);
    signal p1_reg: signed(s_width-1 downto 0);
    signal p: signed(s_width-1 downto 0);
    signal s_sel: signed(s_width-1 downto 0);
    signal a0_reg : signed(a0_width-1 downto 0);
    signal a1_reg : signed(a0_width-1 downto 0);
    signal b0_reg : signed(b0_width-1 downto 0);
    signal b1_reg : signed(b0_width-1 downto 0);
begin
    p0<=a0_reg*b0_reg;
    p1<=a1_reg*b1_reg;
    process(clock,reset)begin
        if(reset='1')then
            a0_reg<=(others=>'0');
            b0_reg<=(others=>'0');
            a1_reg<=(others=>'0');
            b1_reg<=(others=>'0');
        else
            if(rising_edge(clock))then
                if(ce='1')then
                    a0_reg<=a0;
                    b0_reg<=b0;
                    a1_reg<=a1;

```

```
        b1_reg<=b1;
    end if;
end if;
end if;
end process;

process(clock,reset)begin
    if(reset='1')then
        p0_reg<=(others=>'0');
        p1_reg<=(others=>'0');
    else
        if(rising_edge(clock))then
            if(ce='1')then
                p0_reg<=resize(p0,s_width);
                p1_reg<=resize(p1,s_width);
            end if;
        end if;
    end if;
end process;

process(accload,s)begin
    if(accload='1')then
        s_sel<=s;
    else
        s_sel<=(others=>'0');
    end if;
end process;

process(clock,reset)begin
    if(reset='1')then
        s<=(others=>'0');
    else
        if(rising_edge(clock))then
            if(ce='1')then
                s<=s_sel+p0_reg-p1_reg;
            end if;
        end if;
    end if;
end process;
```

```

        end if;
    end if;
end if;
end process;
end Behavioral;

```

10.3.2 动态累加示例

$\Sigma(A*B)$ 的动态选择 (Verilog)

```

// Accumulation select with Dynamic control( $\Sigma(A*B)$ )
// 1 latency stage on operands
// 2 latency stage after the multiplication
module gowin_dsp (a,b,clk,ce,rst,accsel,out);
parameter PRE_LOAD = 48'h00000000000012;
input [25:0]a;
input [16:0]b;
input  clk,ce,rst;
input accsel;
output [42:0]out;
reg [42:0]out_reg;
wire [42:0]s_sel;
reg [42:0]mout;
reg [25:0]a_reg;
reg [16:0]b_reg;
always@(posedge clk)begin
    if (rst)begin
        a_reg<=0;
        b_reg<=0;
    end
    else begin
        if(ce)begin
            a_reg<=a;
            b_reg<=b;
        end
    end
end
end
end

```

```

assign s_sel= accsel == 1'b1 ? out_reg : PRE_LOAD;

always@(posedge clk)begin
    if (rst)begin
        out_reg<=0;
        mout<=0;
    end
    else begin
        if(ce)begin
            mout<=a_reg*b_reg;
            out_reg<=s_sel+mout;
        end
    end
end

assign out=out_reg;
endmodule

```

$\Sigma(A*B)$ 的动态选择 (VHDL)

```

-- Accumulation select with Dynamic control( $\Sigma(A*B)$ )
-- 1 latency stage on operands
-- 2 latency stage after the multiplication
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity gowin_dsp is
    generic(
        a_width : integer := 26;
        b_width : integer := 17;
        dout_width : integer := 43
    );
    port (
        dout: out unsigned (dout_width-1 downto 0);
        a : in unsigned (a_width-1 downto 0);

```

```

        b : in unsigned (b_width-1 downto 0);
        accsel : in std_logic;
        clk,ce,rst:in std_logic
    );
end gowin_dsp;
architecture Behavioral of gowin_dsp is
    signal a_reg : unsigned (a_width-1 downto 0);
    signal b_reg : unsigned (b_width-1 downto 0);
    signal mout,s_sel :unsigned (dout_width-1 downto 0);
    signal out_reg :unsigned (dout_width-1 downto 0);
begin
    process(clk)begin
        if(rising_edge(clk))then
            if(rst='1')then
                a_reg<=(others=>'0');
                b_reg<=(others=>'0');
            else
                if(ce='1')then
                    a_reg<=a;
                    b_reg<=b;
                end if;
            end if;
        end if;
    end process;

    process(accsel,out_reg)begin
        if(accsel='1')then
            s_sel<=out_reg;
        else
            s_sel<="000"&x"0000000012";
        end if;
    end process;

    process(clk)begin
        if(rising_edge(clk))then

```

```

        if(rst='1')then
            out_reg<=(others=>'0');
            mout<=(others=>'0');
        else
            if(ce='1')then
                out_reg<=s_sel + mout;
                mout<=a_reg*b_reg;
            end if;
        end if;
    end if;
end process;

dout<=out_reg;
end Behavioral;

```

$\sum(A0*B0 \pm A1*B1)$ 的动态选择 (Verilog)

```

// Accumulation select with Dynamic control( $\sum(A0*B0 \pm A1*B1)$ )
// 1 latency stage on operands
// 2 latency stage after the multiplication
module gowin_dsp (a0,a1,b0,b1,clk,ce, rst, accsel, dout);
parameter width=12;
parameter widthout=width*2;
parameter PRE_LOAD = 48'h0000000000012;
input signed[width-1:0]a0,a1;
input signed[width-1:0]b0,b1;
input  clk,ce,rst;
input accsel;
output signed[widthout:0]dout;
reg signed[width-1:0]a0_reg,a1_reg;
reg signed[width-1:0]b0_reg,b1_reg;
reg signed[widthout:0]out_reg,fb_reg;
reg signed[widthout-1:0]p0_reg,p1_reg;
wire signed[widthout-1:0]mult0,mult1;
wire signed[widthout:0]acc_sel;
wire signed[widthout:0]add_sub;
reg signed[widthout:0]oreg;

```

```
assign acc_sel = accsel ? oreg : 0;
always@(posedge clk)begin
    if (rst)begin
        a0_reg<=0;
        b0_reg<=0;
        a1_reg<=0;
        b1_reg<=0;
    end
    else begin
        if(ce)begin
            a0_reg<=a0;
            b0_reg<=b0;
            a1_reg<=a1;
            b1_reg<=b1;
        end
    end
end

assign mult0=a0_reg*b0_reg;
assign mult1=a1_reg*b1_reg;

always@(posedge clk)begin
    if (rst)begin
        oreg<=0;
        p0_reg<=0;
        p1_reg<=0;
    end
    else begin
        if(ce)begin
            oreg<=acc_sel+p0_reg+p1_reg;
            p0_reg<=mult0;
            p1_reg<=mult1;
        end
    end
end
```

```

assign dout=oreg;
endmodule

```

$\Sigma(A0*B0 \pm A1*B1)$ 的动态选择 (VHDL)

```

-- Accumulation select with Dynamic control( $\Sigma(A0*B0 \pm A1*B1)$ )
-- 1 latency stage on operands
-- 2 latency stage after the multiplication

```

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
use IEEE.NUMERIC_STD.ALL;
```

```
entity gowin_dsp is
```

```
    generic(
```

```
        a_width : integer := 12;
```

```
        b_width : integer := 12;
```

```
        dout_width : integer := 24
```

```
    );
```

```
    port (
```

```
        dout: out signed (dout_width-1 downto 0);
```

```
        a0,a1 : in signed (a_width-1 downto 0);
```

```
        b0,b1 : in signed (b_width-1 downto 0);
```

```
        accsel : in std_logic;
```

```
        clk,ce,rst:in std_logic
```

```
    );
```

```
end gowin_dsp;
```

```
architecture Behavioral of gowin_dsp is
```

```
    signal a0_reg,a1_reg : signed (a_width-1 downto 0);
```

```
    signal b0_reg,b1_reg : signed (b_width-1 downto 0);
```

```
    signal p0_reg,p1_reg :signed (a_width+b_width-1 downto 0);
```

```
    signal oreg :signed (dout_width-1 downto 0);
```

```
    signal acc_sel:signed (dout_width-1 downto 0);
```

```
begin
```

```
    process(accsel,oreg)begin
```

```
        if(accsel='1')then
```

```
            acc_sel <=  oreg;
```

```
        else
            acc_sel<=(others=>'0');
        end if;
    end process;

process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            a0_reg<=(others=>'0');
            b0_reg<=(others=>'0');
            a1_reg<=(others=>'0');
            b1_reg<=(others=>'0');
        else
            if(ce='1')then
                a0_reg<=a0;
                b0_reg<=b0;
                a1_reg<=a1;
                b1_reg<=b1;
            end if;
        end if;
    end if;
end process;

process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            oreg<=(others=>'0');
            p0_reg<=(others=>'0');
            p1_reg<=(others=>'0');
        else
            if(ce='1')then
                p0_reg<=a0_reg*b0_reg;
                p1_reg<=a1_reg*b1_reg;
                oreg<=acc_sel+p0_reg+p1_reg;
            end if;
        end if;
    end if;
end process;
```

```

        end if;
    end if;
end process;

```

```

    dout<=oreg;
end Behavioral;

```

10.4 预加操作

10.4.1 静态预加/减操作示例

后接乘法器的静态预加（Verilog）

```

// Static pre-adder followed by multiplier
module gowin_dsp(clk,rst,ce,a,b,d,dout);
input [15:0]a;
input [15:0]b;
input [15:0]d;
input clk;
input ce,rst;
output reg [31:0]dout;

reg [15:0]a_reg;
reg [15:0]b_reg;
reg [15:0]d_reg;

always@(posedge clk)begin
    if(rst)begin
        a_reg<=0;
        b_reg<=0;
        d_reg<=0;
    end
    else begin
        if(ce)begin
            a_reg<=a;
            b_reg<=b;
            d_reg<=d;
        end
    end
end

```

```

        end
    end

    always@(posedge clk)begin
        if(rst)begin
            dout<=0;
        end
        else begin
            if(ce)begin
                dout<=(a_reg+d_reg)*b_reg;
            end
        end
    end
end
endmodule

```

后接乘法器的静态预减 (VHDL)

```

-- Static pre-subtract followed by multiplier
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity gowin_dsp is
    generic(
        a_width : integer := 17;
        b_width : integer := 17;
        d_width : integer := 17;
        dout_width : integer := 35
    );
    port (
        dout: out signed (dout_width-1 downto 0);
        a : in signed (a_width-1 downto 0);
        b : in signed (b_width-1 downto 0);
        d : in signed (d_width-1 downto 0);
        clk: in std_logic;
        ce,rst: in std_logic
    );

```

```

end entity;
architecture Behavioral of gowin_dsp is
    signal ad_add:signed (a_width  downto 0);
    signal a_reg : signed (a_width-1 downto 0);
    signal b_reg : signed (b_width-1 downto 0);
    signal d_reg : signed (d_width-1 downto 0);
begin
    process(clk)begin
        if(rising_edge(clk))then
            if(rst='1')then
                a_reg<=(others=>'0');
                b_reg<=(others=>'0');
                d_reg<=(others=>'0');
            else
                if(ce='1')then
                    a_reg<=a;
                    b_reg<=b;
                    d_reg<=d;
                end if;
            end if;
        end if;
    end process;

    ad_add<=resize(a_reg,a_width+1)-resize(d_reg,a_width+1);
    process(clk)begin
        if(rising_edge(clk))then
            if(rst='1')then
                dout<=(others=>'0');
            else
                if(ce='1')then
                    dout<=ad_add*b_reg;
                end if;
            end if;
        end if;
    end process;
end architecture Behavioral of gowin_dsp;

```

```

        end process;
    end Behavioral;

```

后接乘加器的静态预加 (Verilog)

```

// Static pre-adder followed by multiplier and post-adder
module gowin_dsp(clk,rst,ce,a,b,c,d,dout);
    input signed[17:0]a;
    input signed[17:0]b;
    input signed[17:0]d;
    input signed[17:0]c;
    input clk;
    input ce,rst;
    output reg signed[35:0]dout;
    reg signed[35:0]mout;
    reg signed[17:0]a_reg;
    reg signed[17:0]b_reg;
    reg signed[17:0]d_reg;
    reg signed[17:0]c_reg;

    always@(posedge clk)begin
        if(rst)begin
            a_reg<=0;
            b_reg<=0;
            c_reg<=0;
            d_reg<=0;
        end
        else begin
            if(ce)begin
                a_reg<=a;
                b_reg<=b;
                c_reg<=c;
                d_reg<=d;
            end
        end
    end
end

```

```

always@(posedge clk)begin
    if(rst)begin
        dout<=0;
        mout<=0;
    end
    else begin
        if(ce)begin
            mout<=(a_reg+d_reg)*b_reg;
            dout<=mout+c_reg;
        end
    end
end
endmodule

```

后接乘加器的静态预减 (VHDL)

-- Static pre-adder followed by multiplier and post-adder

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

```

```

entity gowin_dsp is
    generic(
        a_width : integer := 17;
        b_width : integer := 17;
        c_width : integer := 17;
        d_width : integer := 17;
        dout_width : integer := 35
    );
    port (
        dout: out signed (dout_width-1 downto 0);
        a : in signed (a_width-1 downto 0);
        b : in signed (b_width-1 downto 0);
        c : in signed (c_width-1 downto 0);
        d : in signed (d_width-1 downto 0);
        clk: in std_logic;
        ce,rst: in std_logic
    );
end entity gowin_dsp;

```

```

    );
end gowin_dsp;

architecture Behavioral of gowin_dsp is
    signal ad_add: signed (a_width  downto 0);
    signal a_reg : signed (a_width-1 downto 0);
    signal b_reg : signed (b_width-1 downto 0);
    signal c_reg : signed (c_width-1 downto 0);
    signal d_reg : signed (d_width-1 downto 0);
    signal mout: signed (dout_width-1 downto 0);
begin
    process(clk)begin
        if(rising_edge(clk))then
            if(rst='1')then
                a_reg<=(others=>'0');
                b_reg<=(others=>'0');
                c_reg<=(others=>'0');
                d_reg<=(others=>'0');
            else
                if(ce='1')then
                    a_reg<=a;
                    b_reg<=b;
                    c_reg<=c;
                    d_reg<=d;
                end if;
            end if;
        end if;
    end process;
    ad_add<=resize(a_reg,18)+resize(d_reg,18);
    process(clk)begin
        if(rising_edge(clk))then
            if(rst='1')then
                dout<=(others=>'0');
                mout<=(others=>'0');
            else
                if(ce='1')then

```

```

        mout<=ad_add*b_reg;
        dout<=mout+c_reg;
    end if;
end if;
end if;
end process;
end Behavioral;

```

10.4.2 动态预加/减操作示例

后接乘法器的动态预加/减 (Verilog)

```

// Pre-add/subtract select with Dynamic control
module gowin_dsp(clk,ce,reset,a,b,d,psel,dout);
input signed [15:0]a;
input signed [15:0]b;
input signed [15:0]d;
input psel;
input [1:0]clk,ce,reset;
output reg signed [31:0]dout;
reg signed [31:0]preg;

reg signed [15:0]a_reg;
reg signed [15:0]b_reg;
reg signed [15:0]d_reg;
always@(posedge clk[0])begin
    if(reset[0])begin
        a_reg<=0;
        b_reg<=0;
        d_reg<=0;
    end
    else begin
        if(ce[0])begin
            a_reg<=a;
            b_reg<=b;
            d_reg<=d;
        end
    end
end

```

```

        end
    end
    always@(posedge clk[1])begin
        if(reset[1])begin
            preg<=0;
        end
        else begin
            if(ce[1])begin
                preg<=psel ? (a_reg-d_reg)*b_reg :
(a_reg+d_reg)*b_reg;
            end
        end
    end
end

always@(posedge clk[1])begin
    if(reset[1])begin
        dout<=0;
    end
    else begin
        if(ce[1])begin
            dout<=preg;
        end
    end
end
endmodule

```

后接乘法器的动态预加/减 (VHDL)

```

-- Pre-add/subtract select with Dynamic control
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity gowin_dsp is
    generic(
        a_width : integer := 16;
        b_width : integer := 16;
        d_width : integer := 16;

```

```

        dout_width : integer := 33
    );
    port (
        dout: out signed (dout_width-1 downto 0);
        a : in signed (a_width-1 downto 0);
        b : in signed (b_width-1 downto 0);
        d : in signed (d_width-1 downto 0);
        psel : in std_logic;
        clk,ce,reset:in signed (1 downto 0)
    );
end entity;

architecture Behavioral of gowin_dsp is
    signal preg:signed (dout_width-1 downto 0);
    signal a_reg : signed (a_width-1 downto 0);
    signal b_reg : signed (b_width-1 downto 0);
    signal d_reg : signed (d_width-1 downto 0);
begin
    process(clk(0))begin
        if(rising_edge(clk(0)))then
            if(reset(0)='1')then
                a_reg<=(others=>'0');
                b_reg<=(others=>'0');
                d_reg<=(others=>'0');
            else
                if(ce(0)='1')then
                    a_reg<=a;
                    b_reg<=b;
                    d_reg<=d;
                end if;
            end if;
        end if;
    end process;

    process(clk(1))begin
        if(rising_edge(clk(1)))then

```

```

        if(reset(1)='1')then
            preg<=(others=>'0');
        else
            if(ce(1)='1')then
                if(psel='0')then
                    preg<=((a_reg(a_width-1)&a_reg) +
(d_reg(d_width-1) &d_reg))*b_reg;
                else
                    preg<=((a_reg(a_width-1)&a_reg) -
(d_reg(d_width-1) &d_reg))*b_reg;
                end if;
            end if;
        end if;
    end if;
end process;

process(clk(1))begin
    if(rising_edge(clk(1)))then
        if(reset(1)='1')then
            dout<=(others=>'0');
        else
            if(ce(1)='1')then
                dout<=preg;
            end if;
        end if;
    end if;
end process;
end Behavioral;

```

后接乘加器的动态预加/减 (Verilog)

```

// Dynamic pre-addition/subtraction followed by multiplier and followed
by adder

module gowin_dsp(clk,ce,reset,a,b,c,d,psel,dout);
    input signed [25:0]a;
    input signed [17:0]b;
    input signed [47:0]c;

```

```
input signed [25:0]d;
input psel;
input [1:0]clk,ce,reset;
output reg signed [47:0]dout;
reg signed [43:0]preg;
reg signed [25:0]a_reg;
reg signed [17:0]b_reg;
reg signed [47:0]c_reg;
reg signed [25:0]d_reg;

always@(posedge clk[0])begin
    if(reset[0])begin
        a_reg<=0;
        b_reg<=0;
        c_reg<=0;
        d_reg<=0;
    end
    else begin
        if(ce[0])begin
            a_reg<=a;
            b_reg<=b;
            c_reg<=c;
            d_reg<=d;
        end
    end
end

always@(posedge clk[1])begin
    if(reset[1])begin
        preg<=0;
    end
    else begin
        if(ce[1])begin
            preg<=psel ? (a_reg-d_reg)*b_reg :
(a_reg+d_reg)*b_reg;
```

```

        end
    end
end

always@(posedge clk[1])begin
    if(reset[1])begin
        dout<=0;
    end
    else begin
        if(ce[1])begin
            dout<=preg+c;
        end
    end
end
endmodule

```

后接乘加器的动态预加/减（VHDL）

--Dynamic pre-addition/subtraction of Signed data followed by multiplier

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity gowin_dsp is
    generic(
        a_width : integer := 26;
        b_width : integer := 18;
        c_width : integer := 48;
        d_width : integer := 26;
        dout_width : integer := 48
    );
    port (
        dout: out signed (dout_width-1 downto 0);
        a : in signed (a_width-1 downto 0);
        b : in signed (b_width-1 downto 0);
        c : in signed (c_width-1 downto 0);

```

```

        d : in signed (d_width-1 downto 0);
        psel : in std_logic;
        clk,ce,reset:in signed (1 downto 0)
    );
end gowin_dsp;
architecture Behavioral of gowin_dsp is
    signal preg:signed (a_width+b_width downto 0);
    signal a_reg : signed (a_width-1 downto 0);
    signal b_reg : signed (b_width-1 downto 0);
    signal c_reg : signed (c_width-1 downto 0);
    signal d_reg : signed (d_width-1 downto 0);
begin
    process(clk(0))begin
        if(rising_edge(clk(0)))then
            if(reset(0)='1')then
                a_reg<=(others=>'0');
                b_reg<=(others=>'0');
                c_reg<=(others=>'0');
                d_reg<=(others=>'0');
            else
                if(ce(0)='1')then
                    a_reg<=a;
                    b_reg<=b;
                    c_reg<=c;
                    d_reg<=d;
                end if;
            end if;
        end if;
    end process;

    process(clk(1))begin
        if(rising_edge(clk(1)))then
            if(reset(1)='1')then
                preg<=(others=>'0');
            else

```

```

        if(ce(1)=='1')then
            if(psel='0')then
                preg<=((a_reg(a_width-1)&a_reg) +
(d_reg(d_width-1) &d_reg))*b_reg;
            else
                preg<=((a_reg(a_width-1)&a_reg) -
(d_reg(d_width-1) &d_reg))*b_reg;
            end if;
        end if;
    end if;
end if;
end process;

process(clk(1))begin
    if(rising_edge(clk(1)))then
        if(reset(1)=='1')then
            dout<=(others=>'0');
        else
            if(ce(1)=='1')then
                dout<=preg+c;
            end if;
        end if;
    end if;
end if;
end process;
end Behavioral;

```

10.5 级联操作

10.5.1 静态级联示例

$A_0*B_0 \pm A_1*B_1 \pm A_2*B_2 \pm A_3*B_3$ (Verilog)

// Static cascade after summing of multipliers ($a_0*b_0 + a_1*b_1 + a_2*b_2 + a_3*b_3$)

// 1 latency stage on operands

// 2 latency stage after the multiplication

module gowin_dsp (a0, a1, a2, b0, b1, b2, a3, b3, s, clock, ce, reset);

parameter a_width=18;

parameter b_width=18;

```

parameter s_width=38;
input signed [a_width-1:0] a0, a1, a2, b0, b1, b2, a3, b3;
input clock, ce, reset;
output signed [s_width-1:0] s;
wire signed [s_width-1:0] p0, p1, p2, p3, s0, s1;
reg signed [s_width-1:0] p0_reg=0;
reg signed [s_width-1:0] p1_reg=0;
reg signed [s_width-1:0] p2_reg=0;
reg signed [s_width-1:0] p3_reg=0;
reg signed [s_width-1:0] s0_reg=0;
reg signed [s_width-1:0] s1_reg=0;
reg signed [a_width-1:0] a0_reg=18'h00000;
reg signed [a_width-1:0] a1_reg=18'h00000;
reg signed [a_width-1:0] a2_reg=18'h00000;
reg signed [a_width-1:0] a3_reg=18'h00000;
reg signed [b_width-1:0] b0_reg=18'h00000;
reg signed [b_width-1:0] b1_reg=18'h00000;
reg signed [b_width-1:0] b2_reg=18'h00000;
reg signed [b_width-1:0] b3_reg=18'h00000;
always @(posedge clock or posedge reset)begin
    if(reset)begin
        a0_reg <= 0;
        a1_reg <= 0;
        a2_reg <= 0;
        a3_reg <= 0;
        b0_reg <= 0;
        b1_reg <= 0;
        b2_reg <= 0;
        b3_reg <= 0;
    end else begin
        if(ce)begin
            a0_reg <= a0;
            a1_reg <= a1;
            a2_reg <= a2;
            a3_reg <= a3;

```

```
        b0_reg <= b0;
        b1_reg <= b1;
        b2_reg <= b2;
        b3_reg <= b3;
    end
end
end

assign p0 = a0_reg*b0_reg;
assign p1 = a1_reg*b1_reg;
assign p2 = a2_reg*b2_reg;
assign p3 = a3_reg*b3_reg;

always @(posedge clock or posedge reset)begin
    if(reset)begin
        p0_reg <= 0;
        p1_reg <= 0;
        p2_reg <= 0;
        p3_reg <= 0;
    end else begin
        if(ce)begin
            p0_reg <= p0;
            p1_reg <= p1;
            p2_reg <= p2;
            p3_reg <= p3;
        end
    end
end

assign s0 = p0_reg + p1_reg;
always @(posedge clock or posedge reset)begin
    if(reset)begin
        s0_reg <= 0;
    end else begin
        if(ce)begin
```

```

        s0_reg <= s0;
    end
end
end
assign s1 = s0_reg + p2_reg + p3_reg;
always @(posedge clock or posedge reset)begin
    if(reset)begin
        s1_reg <= 0;
    end else begin
        if(ce)begin
            s1_reg <= s1;
        end
    end
end
end
assign s=s1_reg;
endmodule

```

$A_0 \cdot B_0 \pm A_1 \cdot B_1 \pm A_2 \cdot B_2 \pm A_3 \cdot B_3$ (VHDL)

-- Static cascade after summing of multipliers($a_0 \cdot b_0 + a_1 \cdot b_1 + d_0 \cdot d_1 + d_2 \cdot d_3$)

-- 1 latency stage on operands

-- 2 latency stage after the multiplication

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity gowin_dsp is

generic(

 a_width : integer := 11;

 b_width : integer := 11;

 dout_width : integer := 23

);

port (

 clk,ce,rst: in std_logic;

 mout: out unsigned (dout_width-1 downto 0);

 a,a1,d0,d1 : in unsigned (a_width-1 downto 0);

 b,b1,d2,d3 : in unsigned (b_width-1 downto 0)

```

    );
end entity;

architecture Behavioral of gowin_dsp is
    signal mult1,mult2,mult3,mult4:unsigned(dout_width-2 downto 0);
    signal a_reg,a1_reg,d0_reg,d1_reg : unsigned (a_width-1
downto 0);
    signal b_reg,b1_reg,d2_reg,d3_reg : unsigned (b_width-1
downto 0);
    signal p0_reg,p1_reg : unsigned (dout_width-2 downto 0);
begin
    process(clk)begin
        if(rising_edge(clk))then
            if(rst='1')then
                a_reg <=(others=>'0');
                a1_reg<=(others=>'0');
                b_reg <=(others=>'0');
                b1_reg<=(others=>'0');
                d0_reg<=(others=>'0');
                d1_reg<=(others=>'0');
                d2_reg<=(others=>'0');
                d3_reg<=(others=>'0');
            else
                if(ce='1')then
                    a_reg <=a;
                    a1_reg<=a1;
                    b_reg <=b;
                    b1_reg<=b1;
                    d0_reg<=d0;
                    d1_reg<=d1;
                    d2_reg<=d2;
                    d3_reg<=d3;
                end if;
            end if;
        end if;
    end process;
end process;

```

```

process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            mout<=(others=>'0');
            p0_reg<=(others=>'0');
            p1_reg<=(others=>'0');
        else
            if(ce='1')then
                mout<=(p0_reg(dout_width-2) & p0_reg) +
(p1_reg(dout_width-2) & p1_reg);
                p0_reg<=a_reg*b_reg + d0_reg*d1_reg;
                p1_reg<=a1_reg*b1_reg + d2_reg*d3_reg;
            end if;
        end if;
    end if;
end process;
end Behavioral;

```

A0*B0±A1*B1 (Verilog)

```

// Static cascading of multipliers( a*b+a1*b1)
// 1 latency stage on operands
// 2 latency stage after the multiplication
module gowin_dsp(clk,ce,rst,a,b,a1,b1,dout);
parameter widtha=20;
parameter widthb=18;
parameter widthout=widtha+widthb;
input signed[widtha-1:0]a,a1;
input signed[widthb-1:0]b,b1;
input clk,ce,rst;
output reg signed[widthout-1:0]dout;
reg signed[widtha-1:0]a_reg,a1_reg;
reg signed[widthb-1:0]b_reg,b1_reg;
reg signed[widthout-1:0]p0_reg,p1_reg;

always@(posedge clk)begin

```

```
    if(rst)begin
        a_reg <= 0;
        a1_reg <= 0;
        b_reg <= 0;
        b1_reg <= 0;
    end
    else begin
        if(ce)begin
            a_reg <= a;
            a1_reg <= a1;
            b_reg <= b;
            b1_reg <= b1;
        end
    end
end

always@(posedge clk)begin
    if(rst)begin
        p0_reg <= 0;
        p1_reg <= 0;
    end
    else begin
        if(ce)begin
            p0_reg <= a_reg*b_reg;
            p1_reg <= a1_reg*b1_reg;
        end
    end
end

always@(posedge clk)begin
    if(rst)begin
        dout <= 0;
    end
    else begin
        if(ce)begin
```

```

        dout<=p0_reg+p1_reg;
    end
end
endmodule

```

A0*B0±A1*B1 (VHDL)

```

-- Static cascading of multipliers(a*b+a1*b1)
-- 1 latency stage on operands
-- 2 latency stage after the multiplication
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity gowin_dsp is
    generic(
        a_width : integer := 27;
        b_width : integer := 18;
        dout_width : integer := 46
    );
    port (
        dout: out signed (dout_width-1 downto 0);
        a,a1 : in signed (a_width-1 downto 0);
        b,b1 : in signed (b_width-1 downto 0);
        clk,ce,rst:in std_logic
    );
end gowin_dsp;

architecture Behavioral of gowin_dsp is
    signal a_reg,a1_reg : signed (a_width-1 downto 0);
    signal b_reg,b1_reg : signed (b_width-1 downto 0);
    signal p0_reg,p1_reg :signed (dout_width -2 downto 0);
begin
    process(clk)begin
        if(rising_edge(clk))then
            if(rst='1')then
                a_reg<=(others=>'0');

```

```

        a1_reg<=(others=>'0');
        b_reg<=(others=>'0');
        b1_reg<=(others=>'0');
    else
        if(ce='1')then
            a_reg<=a;
            a1_reg<=a1;
            b_reg<=b;
            b1_reg<=b1;
        end if;
    end if;
end if;
end process;

process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            p0_reg<=(others=>'0');
            p1_reg<=(others=>'0');
        else
            if(ce='1')then
                p0_reg<=a_reg*b_reg;
                p1_reg<=a1_reg*b1_reg;
            end if;
        end if;
    end if;
end process;

process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            dout<=(others=>'0');
        else
            if(ce='1')then
                dout<=(p0_reg(dout_width -2) & p0_reg) +

```

```

        (p1_reg(dout_width -2) & p1_reg);
        end if;
        end if;
        end if;
    end process;
end Behavioral;

```

10.5.2 动态级联示例

A0*B0±A1*B1 的动态选择 (Verilog)

```

// Cascade select with Dynamic control(a*b+a1*b1))
// 1 latency stage on operands
// 2 latency stage after the multiplication
module gowin_dsp(clk,ce,rst,sel,a,b,a1,b1,out);
parameter widtha=26;
parameter widthb=17;
parameter widthout=widtha+widthb;

input [widtha-1:0]a,a1;
input [widthb-1:0]b,b1;
input clk,ce,rst;
input sel;
output reg [widthout:0]out;
reg [widtha-1:0]a_reg,a1_reg;
reg [widthb-1:0]b_reg,b1_reg;
reg [widthout-1:0]p0_reg,p1_reg;

always@(posedge clk)begin
    if(rst)begin
        a_reg <= 0;
        a1_reg <= 0;
        b_reg <= 0;
        b1_reg <= 0;
    end
    else begin
        if(ce)begin

```

```

        a_reg <= a;
        a1_reg <= a1;
        b_reg <= b;
        b1_reg <= b1;
    end
end
end

always@(posedge clk)begin
    if(rst)begin
        p0_reg <= 0;
        p1_reg <= 0;
    end
    else begin
        if(ce)begin
            p0_reg <= a_reg*b_reg;
            p1_reg <= a1_reg*b1_reg;
        end
    end
end

always@(posedge clk)begin
    if(rst)begin
        out <= 0;
    end
    else begin
        if(ce)begin
            out <= sel ? p0_reg + p1_reg : p1_reg;
        end
    end
end
endmodule

```

A0*B0±A1*B1 的动态选择 (VHDL)

```

-- Cascade select with Dynamic control(a*b+a1*b1))
-- 1 latency stage on operands

```

```

-- 2 latency stage after the multiplication
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity gowin_dsp is
    generic(
        a_width : integer := 26;
        b_width : integer := 17;
        dout_width : integer := 43
    );
    port (
        dout: out unsigned (dout_width-1 downto 0);
        a,a1 : in unsigned (a_width-1 downto 0);
        b,b1 : in unsigned (b_width-1 downto 0);
        sel : in std_logic;
        clk,ce,rst:in std_logic
    );
end entity;

architecture Behavioral of gowin_dsp is
    signal a_reg,a1_reg : unsigned (a_width-1 downto 0);
    signal b_reg,b1_reg : unsigned (b_width-1 downto 0);
    signal p0_reg,p1_reg :unsigned (dout_width -1 downto 0);
begin
    process(clk)begin
        if(rising_edge(clk))then
            if(rst='1')then
                a_reg<=(others=>'0');
                a1_reg<=(others=>'0');
                b_reg<=(others=>'0');
                b1_reg<=(others=>'0');
            else
                if(ce='1')then
                    a_reg<=a;
                    a1_reg<=a1;

```

```

        b_reg<=b;
        b1_reg<=b1;
    end if;
end if;
end if;
end process;

process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            p0_reg<=(others=>'0');
            p1_reg<=(others=>'0');
        else
            if(ce='1')then
                p0_reg<=a_reg*b_reg;
                p1_reg<=a1_reg*b1_reg;
            end if;
        end if;
    end if;
end process;

process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            dout<=(others=>'0');
        else
            if(ce='1')then
                if(sel='1')then
                    dout<=p0_reg+p1_reg;
                else
                    dout<=p1_reg;
                end if;
            end if;
        end if;
    end if;
end process;

```

```

        end process;
    end Behavioral;

```

A0*B0±A1*B1±A2*B2±A3*B3 的动态选择 (Verilog)

```

// Cascade select with Dynamic control(a0*b0±a1*b1± a2*b2±a3*b3))
// 1 latency stage on operands
// 2 latency stage after the multiplication
module gowin_dsp(clk,ce,rst,sel,a0,a1,d0, d1,b0, b1, d2, d3,dout);
parameter widtha=12;
parameter widthb=12;
parameter widthout=widtha+widthb;

input signed[widtha-1:0]a0,a1,d0,d1;
input signed[widthb-1:0]b0,b1,d2,d3;
input clk,ce,rst;
input sel;
output reg signed[widthout:0]dout;
reg signed[widtha-1:0]a0_reg,a1_reg,d0_reg,d1_reg;
reg signed[widthb-1:0]b0_reg,b1_reg,d2_reg,d3_reg;
reg signed[widthout-1:0]p0_reg,p1_reg,p2_reg,p3_reg;

always@(posedge clk)begin
    if(rst)begin
        a0_reg <= 0;
        a1_reg <= 0;
        b0_reg <= 0;
        b1_reg <= 0;
        d0_reg <= 0;
        d1_reg <= 0;
        d2_reg <= 0;
        d3_reg <= 0;
    end
    else begin
        if(ce)begin
            a0_reg <= a0;
            a1_reg <= a1;

```

```
        b0_reg <= b0;
        b1_reg <= b1;
        d0_reg <= d0;
        d1_reg <= d1;
        d2_reg <= d2;
        d3_reg <= d3;
    end
end
end

always@(posedge clk)begin
    if(rst)begin
        p0_reg <= 0;
        p1_reg <= 0;
    end
    else begin
        if(ce)begin
            p0_reg <= a0_reg*b0_reg+a1_reg*b1_reg;
            p1_reg <= d0_reg*d1_reg+d2_reg*d3_reg;
        end
    end
end

always@(posedge clk)begin
    if(rst)begin
        dout <= 0;
    end
    else begin
        if(ce)begin
            dout <= sel ? p0_reg + p1_reg : p0_reg ;
        end
    end
end
endmodule
```

A0*B0±A1*B1±A2*B2±A3*B3 的动态选择 (VHDL)

```
-- Cascade select with Dynamic control(a*b±a1*b1± d0*d1±d2*d3))
-- 1 latency stage on operands
-- 2 latency stage after the multiplication
```

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
use IEEE.NUMERIC_STD.ALL;
```

```
entity gowin_dsp is
```

```
  generic(
```

```
    a_width : integer := 11;
```

```
    b_width : integer := 11;
```

```
    dout_width : integer := 23
```

```
  );
```

```
  port (
```

```
    clk,ce,rst,sel: in std_logic;
```

```
    mout: out unsigned (dout_width-1 downto 0);
```

```
    a,a1,d0,d1 : in unsigned (a_width-1 downto 0);
```

```
    b,b1,d2,d3 : in unsigned (b_width-1 downto 0)
```

```
  );
```

```
end entity ;
```

```
architecture Behavioral of gowin_dsp is
```

```
  signal mult1,mult2,mult3,mult4:unsigned(dout_width-2 downto 0);
```

```
  signal a_reg,a1_reg,d0_reg,d1_reg : unsigned (a_width-1
downto 0);
```

```
  signal b_reg,b1_reg,d2_reg,d3_reg : unsigned (b_width-1
downto 0);
```

```
  signal p0_reg,p1_reg : unsigned (dout_width-2 downto 0);
```

```
begin
```

```
  process(clk)begin
```

```
    if(rising_edge(clk))then
```

```
      if(rst='1')then
```

```
        a_reg <=(others=>'0');
```

```
        a1_reg<=(others=>'0');
```

```
        b_reg <=(others=>'0');
```

```

        b1_reg<=(others=>'0');
        d0_reg<=(others=>'0');
        d1_reg<=(others=>'0');
        d2_reg<=(others=>'0');
        d3_reg<=(others=>'0');
    else
        if(ce='1')then
            a_reg <=a;
            a1_reg<=a1;
            b_reg <=b;
            b1_reg<=b1;
            d0_reg<=d0;
            d1_reg<=d1;
            d2_reg<=d2;
            d3_reg<=d3;
        end if;
    end if;
end if;
end process;

process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            p0_reg<=(others=>'0');
            p1_reg<=(others=>'0');
        else
            if(ce='1')then
                p0_reg<=a_reg*b_reg + d0_reg*d1_reg;
                p1_reg<=a1_reg*b1_reg + d2_reg*d3_reg;
            end if;
        end if;
    end if;
end process;

process(clk)begin

```

```

        if(rising_edge(clk))then
            if(rst='1')then
                mout<=(others=>'0');
            else
                if(ce='1')then
                    if(sel='1')then
                        mout<=(p0_reg(dout_width-2) & p0_reg) +
(p1_reg(dout_width-2) & p1_reg);
                    else
                        mout<=(p0_reg(dout_width-2) & p0_reg);
                    end if;
                end if;
            end if;
        end if;
    end process;
end Behavioral;

```

10.6 移位操作

10.6.1 移位操作示例

预加器后加乘法器的移位（Verilog）

```

// Unsigned pre-adder followed by shift of multiplier
// 1 latency stage on operands
// 2 latency stage after the multiplication
module gowin_dsp(a0, a1, b0, b1, p0, p1, clk, ce, reset);
    parameter a_width=18;
    parameter b_width=18;
    parameter p_width=36;
    input [a_width-1:0] a0, a1;
    input [b_width-1:0] b0, b1;
    input clk, ce, reset;
    output reg[p_width-1:0] p0, p1;
    wire [b_width-1:0] b0_padd, b1_padd;
    reg [b_width-1:0] b0_reg=18'h00000;
    reg [b_width-1:0] b1_reg=18'h00000;
    reg [b_width-1:0] bX1=18'h00000;

```

```
reg [b_width-1:0] bY1=18'h00000;
reg[p_width-1:0] p0_reg, p1_reg;
always @(posedge clk or posedge reset)
begin
    if(reset)begin
        b0_reg <= 0;
        b1_reg <= 0;
        bX1 <= 0;
        bY1 <= 0;
    end else begin
        if(ce)begin
            b0_reg <= b0;
            b1_reg <= b1;
            bX1 <= b0_reg;
            bY1 <= b1_reg;
        end
    end
end
assign b0_padd = bX1 + b1_reg;
assign b1_padd= b0_reg + bY1;

always @(posedge clk or posedge reset)
begin
    if(reset)begin
        p0_reg <= 0;
        p1_reg <= 0;
    end else begin
        if(ce)begin
            p0_reg <= a0 * b0_padd;
            p1_reg <= a1 * b1_padd;
        end
    end
end
always @(posedge clk or posedge reset)
begin
```

```

    if(reset)begin
        p0 <= 0;
        p1 <= 0;
    end else begin
        if(ce)begin
            p0<=p0_reg;
            p1<=p1_reg;
        end
    end
end
endmodule

```

预加器后加乘法器的移位 (VHDL)

```

-- Unsigned pre-adder followed by shift of multiplier
-- 1 latency stage on operands
-- 2 latency stage after the multiplication

```

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

```

```

entity gowin_dsp is

```

```

    generic(
        a_width : integer := 26;
        b_width : integer := 18;
        dout_width : integer := 45
    );

```

```

    port (
        p0,p1: out signed (dout_width-1 downto 0);
        a,d0,d1 : in signed (a_width-1 downto 0);
        b0,b1 : in signed (b_width-1 downto 0);
        clk,ce,rst:in std_logic
    );

```

```

end entity;

```

```

architecture Behavioral of gowin_dsp is

```

```

    signal a_reg0,a_reg1 ,a_reg2,d0_reg,d1_reg: signed (a_width-1
downto 0);

```

```

    signal paddsub0,paddsub1 :signed (a_width downto 0);
    signal p0_reg,p1_reg:   signed (dout_width-1 downto 0);
begin
    process(clk)begin
        if(rising_edge(clk))then
            if(rst='1')then
                a_reg0<=(others=>'0');
                a_reg1<=(others=>'0');
                a_reg2<=(others=>'0');
                d0_reg<=(others=>'0');
                d1_reg<=(others=>'0');
            else
                if(ce='1')then
                    a_reg0<=a;
                    a_reg1<=a_reg0;
                    a_reg2<=a_reg1;
                    d0_reg<=d0;
                    d1_reg<=d1;
                end if;
            end if;
        end if;
    end process;

    paddsub0<=(a_reg0(a_width-1) & a_reg0) + (d0_reg (a_width-1)
& d0_reg);
    paddsub1<=(a_reg2(a_width-1) & a_reg2) + (d1_reg (a_width-1)
& d1_reg);

    process(clk)begin
        if(rising_edge(clk))then
            if(rst='1')then
                p0_reg<=(others=>'0');
                p1_reg<=(others=>'0');
            else
                if(ce='1')then

```

```

        p0_reg<=paddsub0 * b0;
        p1_reg<=paddsub1 * b1;
    end if;
end if;
end if;
end process;

process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            p0<=(others=>'0');
            p1<=(others=>'0');
        else
            if(ce='1')then
                p0<=p0_reg;
                p1<=p1_reg;
            end if;
        end if;
    end if;
end if;
end process;

end Behavioral;

```

10.7 算术逻辑操作

10.7.1 算数逻辑示例

48 位的加法 (Verilog)

```

// 48-bit wide addition operation.
// 1 latency stage on operands
// 1 latency stage after the sum
module gowin_dsp(a, b, dout, clk, ce, reset);
    parameter width=48;
    input [width-1:0] a, b;
    input clk, ce, reset;
    output reg [width-1:0] dout;
    wire [width-1:0] s_sel;

```

```

reg [width-1:0] a_reg;
reg [width-1:0] b_reg;

always @(posedge clk or posedge reset)
begin
    if(reset)begin
        a_reg <= 0;
        b_reg <= 0;
    end else begin
        if(ce)begin
            a_reg <= a;
            b_reg <= b;
        end
    end
end

always @(posedge clk or posedge reset)
begin
    if(reset)begin
        dout <= 0;
    end else begin
        if(ce)begin
            dout <= a_reg + b_reg;
        end
    end
end
endmodule

```

48 位的减法 (VHDL)

```

-- 48-bit subtraction
-- 1 latency stage on operands
-- 1 latency stage after the sum
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

```

```

entity gowin_dsp is
  generic(
    b_width: integer := 48;
    a_width : integer := 48;
    dout_width : integer := 48
  );
  port (
    dout: out std_logic_vector(dout_width-1 downto 0);
    a: in std_logic_vector(a_width-1 downto 0);
    b: in std_logic_vector(b_width-1 downto 0);
    ce: in std_logic;
    clk: in std_logic;
    reset: in std_logic
  );
end gowin_dsp;

architecture Behavioral of gowin_dsp is
  signal a_reg: std_logic_vector(a_width-1 downto 0);
  signal b_reg: std_logic_vector(b_width-1 downto 0);
begin
  process(clk,reset)begin
    if(reset='1')then
      a_reg<=(others=>'0');
      b_reg<=(others=>'0');
    else
      if(rising_edge(clk))then
        if(ce='1')then
          a_reg<=a;
          b_reg<=b;
        end if;
      end if;
    end if;
  end process;

  process(clk,reset)begin
    if(reset='1')then

```

```
        dout<=(others=>'0');  
    else  
        if(rising_edge(clk))then  
            if(ce='1')then  
                dout<=a_reg - b_reg;  
            end if;  
        end if;  
    end if;  
end process;  
end Behavioral;
```

11_{FSM}

11.1 FSM 描述

GowinSynthesis 支持 Moore 和 Mealy 形式的有限状态机 (FSM) 规范。
FSM 由以下部分组成：

- 状态寄存器
- 下一状态函数
- 输出函数

Moore 形式的有限状态机：输出仅取决于当前状态（与输入变化无关）。

Mealy 形式的有限状态机：输出同时取决于当前状态和输入信号。

11.2 编码方式

- 独热码：每个状态使用一个独立的触发器表示，即对于 N 个状态的状态机，使用 N 位编码，每个状态对应一位为"1"，其余位为"0"。
- 格雷码：相邻状态之间只有一位发生变化
- 顺序编码：使用最少的位数 ($\log_2(N)$) 来表示 N 个状态，状态按二进制顺序编码

11.3 FSM 示例

11.3.1 独热码编码方式的 Moore 型状态机 (Verilog)

```
// State Machine with One-hot encoding
module gowin_fsm (clk,rst,sel,qout);
input clk;
input rst;
input [1:0]sel;
output reg [7:0] qout;
localparam S0 = 6'b00_0001;
```

```
localparam S1 = 6'b00_0010;
localparam S2 = 6'b00_0100;
localparam S3 = 6'b00_1000;
localparam S4 = 6'b01_0000;
localparam S5 = 6'b10_0000;

reg [5:0] curr_state;
reg [5:0] next_state;
always @(posedge clk or posedge rst) begin
    if(rst)
        curr_state <= S0;
    else
        curr_state <= next_state;
end

always @(*) begin
    next_state = S0;
    case(curr_state)
        S0 :begin
            if(sel == 2'b00)
                next_state = S1;
            else
                next_state = S0;
        end

        S1 :begin
            if(sel == 2'b01)
                next_state = S2;
            else
                next_state = S1;
        end

        S2 :begin
            if(sel == 2'b10)
                next_state = S3;
```

```
        else
            next_state = S2;
        end

    S3 :begin
        if(sel == 2'b11)
            next_state = S4;
        else
            next_state = S3;
        end
    S4 :begin
        if(sel == 2'b00)
            next_state = S4;
        else
            next_state = S5;
        end
    S5 :begin
        if(sel == 2'b01)
            next_state = S5;
        else
            next_state = S0;
        end
        default:next_state = S0;
    endcase
end
always @(posedge clk or posedge rst) begin
    if(rst) begin
        qout <= 0;
    end
    else begin
        if(curr_state==S5)begin
            qout <= 8'h10;
        end
    end
end
end
```

```
endmodule
```

11.3.2 独热码编码方式的 Moore 型状态机（VHDL）

```
-- State Machine with One-hot encoding
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity gowin_fsm is
    port(
        clk : in std_logic;
        rst : in std_logic;
        sel : in std_logic_vector(1 downto 0);
        qout: out std_logic_vector(7 downto 0)
    );
end entity;

architecture rtl of gowin_fsm is
    signal curr_state:std_logic_vector(5 downto 0);
    signal next_state:std_logic_vector(5 downto 0);
    constant S0 : std_logic_vector := "000001";
    constant S1 : std_logic_vector := "000010";
    constant S2 : std_logic_vector := "000100";
    constant S3 : std_logic_vector := "001000";
    constant S4 : std_logic_vector := "010000";
    constant S5 : std_logic_vector := "100000";
begin
    process(clk,rst)begin
        if(rst='1')then
            curr_state<=S0;
        else
            if(rising_edge(clk))then
                curr_state<=next_state;
            end if;
        end if;
    end process;
end architecture;
```

```
end process;
```

```
process(sel,curr_state)begin
  next_state <= S0;
  case curr_state is
    when S0 =>
      if(sel ="00")then
        next_state <= S1;
      else
        next_state <= S0;
      end if;
    when S1 =>
      if(sel ="01")then
        next_state <= S2;
      else
        next_state <= S1;
      end if;
    when S2 =>
      if(sel ="10")then
        next_state <= S3;
      else
        next_state <= S2;
      end if;
    when S3 =>
      if(sel ="11")then
        next_state <= S4;
      else
        next_state <= S3;
      end if;
    when S4 =>
      if(sel ="00")then
        next_state <= S4;
      else
        next_state <= S5;
      end if;
    when S5 =>
      if(sel ="00")then
        next_state <= S4;
      else
        next_state <= S5;
      end if;
  end case;
end process;
```

```

        end if;
    when S5 =>
        if(sel ="01")then
            next_state <= S5;
        else
            next_state <= S0;
        end if;
    when others=>next_state <= S0;
end case;
end process;

process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            qout<=(others=>'0');
        else
            if(curr_state =S5)then
                qout <= "00010000";
            end if;
        end if;
    end if;
end if;
end process;
end rtl;

```

11.3.3 独热码编码方式的 Mealy 型状态机（Verilog）

```

// State Machine with One-hot encoding
module gowin_fsm (clk,rst,sel,din,qout);
    input clk;
    input rst;
    input [1:0]sel;
    input [7:0] din;
    output reg [7:0] qout;
    localparam S0 = 6'b00_0001;
    localparam S1 = 6'b00_0010;
    localparam S2 = 6'b00_0100;

```

```
localparam S3 = 6'b00_1000;
localparam S4 = 6'b01_0000;
localparam S5 = 6'b10_0000;

reg [5:0] curr_state;
reg [5:0] next_state;
always @(posedge clk or posedge rst) begin
    if(rst)
        curr_state <= S0;
    else
        curr_state <= next_state;
end

always @(*) begin
    next_state = S0;
    case(curr_state)
        S0 :begin
            if(sel == 2'b00)
                next_state = S1;
            else
                next_state = S0;
            end

        S1 :begin
            if(sel == 2'b01)
                next_state = S2;
            else
                next_state = S1;
            end

        S2 :begin
            if(sel == 2'b10)
                next_state = S3;
            else
                next_state = S2;
            end
    endcase
end
```

```
        end

        S3 :begin
            if(sel == 2'b11)
                next_state = S4;
            else
                next_state = S3;
            end
        S4 :begin
            if(sel == 2'b00)
                next_state = S4;
            else
                next_state = S5;
            end
        S5 :begin
            if(sel == 2'b01)
                next_state = S5;
            else
                next_state = S0;
            end
            default:next_state = S0;
        endcase
    end
    always @(posedge clk or posedge rst) begin
        if(rst) begin
            qout <= 0;
        end
        else begin
            if(curr_state==S5)begin
                qout <= din;
            end
            else begin
                qout <= 8'h10;
            end
        end
    end
end
```

```
end
```

```
endmodule
```

11.3.4 独热码编码方式的 Mealy 型状态机 (VHDL)

```
-- State Machine with One-hot encoding
```

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
use ieee.std_logic_unsigned.all;
```

```
entity gowin_fsm is
```

```
  port(
```

```
    clk : in std_logic;
```

```
    rst : in std_logic;
```

```
    sel : in std_logic_vector(1 downto 0);
```

```
    din : in std_logic_vector(7 downto 0);
```

```
    qout: out std_logic_vector(7 downto 0)
```

```
  );
```

```
end entity;
```

```
architecture rtl of gowin_fsm is
```

```
  signal curr_state:std_logic_vector(5 downto 0);
```

```
  signal next_state:std_logic_vector(5 downto 0);
```

```
  constant S0 : std_logic_vector := "000001";
```

```
  constant S1 : std_logic_vector := "000010";
```

```
  constant S2 : std_logic_vector := "000100";
```

```
  constant S3 : std_logic_vector := "001000";
```

```
  constant S4 : std_logic_vector := "010000";
```

```
  constant S5 : std_logic_vector := "100000";
```

```
begin
```

```
  process(clk,rst)begin
```

```
    if(rst='1')then
```

```
      curr_state<=S0;
```

```
    else
```

```
      if(rising_edge(clk))then
```

```
        curr_state<=next_state;
    end if;
end if;
end process;
```

```
process(sel,curr_state)begin
    next_state <= S0;
    case curr_state is
        when S0 =>
            if(sel ="00")then
                next_state <= S1;
            else
                next_state <= S0;
            end if;
        when S1 =>
            if(sel ="01")then
                next_state <= S2;
            else
                next_state <= S1;
            end if;
        when S2 =>
            if(sel ="10")then
                next_state <= S3;
            else
                next_state <= S2;
            end if;
        when S3 =>
            if(sel ="11")then
                next_state <= S4;
            else
                next_state <= S3;
            end if;
        when S4 =>
            if(sel ="00")then
```

```

        next_state <= S4;
    else
        next_state <= S5;
    end if;
when S5 =>
    if(sel ="01")then
        next_state <= S5;
    else
        next_state <= S0;
    end if;
    when others=>next_state <= S0;
end case;
end process;

process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            qout<=(others=>'0');
        else
            if(curr_state =S5)then
                qout <=din;
            else
                qout <= "00010000";
            end if;
        end if;
    end if;
end process;
end rtl;

```

11.3.5 格雷码编码方式的 Moore 型状态机（Verilog）

```

// State Machine with Gray code encoding
module gowin_fsm (clk,rst,sel,qout);
    input clk;
    input rst;
    input [1:0]sel;

```

```
output reg [7:0] qout;
localparam S0 = 3'b000;
localparam S1 = 3'b001;
localparam S2 = 3'b011;
localparam S3 = 3'b010;
localparam S4 = 3'b110;
localparam S5 = 3'b111;

reg [2:0] curr_state;
reg [2:0] next_state;
always @(posedge clk or posedge rst) begin
    if(rst)
        curr_state <= S0;
    else
        curr_state <= next_state;
end

always @(*) begin
    next_state = S0;
    case(curr_state)
        S0 :begin
            if(sel == 2'b00)
                next_state = S1;
            else
                next_state = S0;
            end

        S1 :begin
            if(sel == 2'b01)
                next_state = S2;
            else
                next_state = S1;
            end

        S2 :begin
```

```
        if(sel == 2'b10)
            next_state = S3;
        else
            next_state = S2;
        end

    S3 :begin
        if(sel == 2'b11)
            next_state = S4;
        else
            next_state = S3;
        end
    S4 :begin
        if(sel == 2'b00)
            next_state = S4;
        else
            next_state = S5;
        end
    S5 :begin
        if(sel == 2'b01)
            next_state = S5;
        else
            next_state = S0;
        end
        default:next_state = S0;
    endcase
end
always @(posedge clk or posedge rst) begin
    if(rst) begin
        qout <= 0;
    end
    else begin
        if(curr_state==S5)begin
            qout <= 8'h10;
        end
    end
end
```

```

        end
    end

```

```

endmodule

```

11.3.6 格雷码编码方式的 Moore 型状态机 (VHDL)

```

-- State Machine with Gray code encoding
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity gowin_fsm is
    port(
        clk : in std_logic;
        rst : in std_logic;
        sel : in std_logic_vector(1 downto 0);
        qout: out std_logic_vector(7 downto 0)
    );
end entity;

architecture rtl of gowin_fsm is
    signal curr_state:std_logic_vector(2 downto 0);
    signal next_state:std_logic_vector(2 downto 0);
    constant S0 : std_logic_vector := "000";
    constant S1 : std_logic_vector := "001";
    constant S2 : std_logic_vector := "011";
    constant S3 : std_logic_vector := "010";
    constant S4 : std_logic_vector := "110";
    constant S5 : std_logic_vector := "111";
begin
    process(clk,rst)begin
        if(rst='1')then
            curr_state<=S0;
        else
            if(rising_edge(clk))then

```

```
        curr_state<=next_state;
    end if;
end if;
end process;

process(sel,curr_state)begin
    next_state <= S0;
    case curr_state is
        when S0 =>
            if(sel ="00")then
                next_state <= S1;
            else
                next_state <= S0;
            end if;
        when S1 =>
            if(sel ="01")then
                next_state <= S2;
            else
                next_state <= S1;
            end if;
        when S2 =>
            if(sel ="10")then
                next_state <= S3;
            else
                next_state <= S2;
            end if;
        when S3 =>
            if(sel ="11")then
                next_state <= S4;
            else
                next_state <= S3;
            end if;
        when S4 =>
            if(sel ="00")then
                next_state <= S4;
```

```

        else
            next_state <= S5;
        end if;
    when S5 =>
        if(sel ="01")then
            next_state <= S5;
        else
            next_state <= S0;
        end if;
    when others=>next_state <= S0;
end case;
end process;

process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            qout<=(others=>'0');
        else
            if(curr_state =S5)then
                qout <= "00010000";
            end if;
        end if;
    end if;
end process;
end rtl;

```

11.3.7 格雷码编码方式的 Mealy 型状态机（Verilog）

```

// State Machine with Gray code encoding
module gowin_fsm (clk,rst,sel,din,qout);
    input clk;
    input rst;
    input [1:0]sel;
    input [7:0] din;
    output reg [7:0] qout;
    localparam S0 = 3'b000;

```

```
localparam S1 = 3'b001;
localparam S2 = 3'b011;
localparam S3 = 3'b010;
localparam S4 = 3'b110;
localparam S5 = 3'b111;
reg [2:0] curr_state;
reg [2:0] next_state;
always @(posedge clk or posedge rst) begin
    if(rst)
        curr_state <= S0;
    else
        curr_state <= next_state;
end

always @(*) begin
    next_state = S0;
    case(curr_state)
        S0 :begin
            if(sel == 2'b00)
                next_state = S1;
            else
                next_state = S0;
        end

        S1 :begin
            if(sel == 2'b01)
                next_state = S2;
            else
                next_state = S1;
        end

        S2 :begin
            if(sel == 2'b10)
                next_state = S3;
            else
```

```
        next_state = S2;
    end

    S3 :begin
        if(sel == 2'b11)
            next_state = S4;
        else
            next_state = S3;
        end
    S4 :begin
        if(sel == 2'b00)
            next_state = S4;
        else
            next_state = S5;
        end
    S5 :begin
        if(sel == 2'b01)
            next_state = S5;
        else
            next_state = S0;
        end
        default:next_state = S0;
    endcase
end
always @(posedge clk or posedge rst) begin
    if(rst) begin
        qout <= 0;
    end
    else begin
        if(curr_state==S5)begin
            qout <= din;
        end
        else begin
            qout <= 8'h10;
        end
    end
end
```

```

        end
    end

```

```

endmodule

```

11.3.8 格雷码编码方式的 Mealy 型状态机 (VHDL)

```

-- State Machine with Gray code encoding
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity gowin_fsm is
    port(
        clk : in std_logic;
        rst : in std_logic;
        sel : in std_logic_vector(1 downto 0);
        din : in std_logic_vector(7 downto 0);
        qout: out std_logic_vector(7 downto 0)
    );
end entity;

architecture rtl of gowin_fsm is
    signal curr_state:std_logic_vector(2 downto 0);
    signal next_state:std_logic_vector(2 downto 0);
    constant S0 : std_logic_vector := "000";
    constant S1 : std_logic_vector := "001";
    constant S2 : std_logic_vector := "011";
    constant S3 : std_logic_vector := "010";
    constant S4 : std_logic_vector := "110";
    constant S5 : std_logic_vector := "111";
begin
    process(clk,rst)begin
        if(rst='1')then
            curr_state<=S0;
        else

```

```
        if(rising_edge(clk))then
            curr_state<=next_state;
        end if;
    end if;
end process;
```

```
process(sel,curr_state)begin
    next_state <= S0;
    case curr_state is
        when S0 =>
            if(sel ="00")then
                next_state <= S1;
            else
                next_state <= S0;
            end if;
        when S1 =>
            if(sel ="01")then
                next_state <= S2;
            else
                next_state <= S1;
            end if;
        when S2 =>
            if(sel ="10")then
                next_state <= S3;
            else
                next_state <= S2;
            end if;
        when S3 =>
            if(sel ="11")then
                next_state <= S4;
            else
                next_state <= S3;
            end if;
        when S4 =>
```

```

        if(sel ="00")then
            next_state <= S4;
        else
            next_state <= S5;
        end if;
    when S5 =>
        if(sel ="01")then
            next_state <= S5;
        else
            next_state <= S0;
        end if;
    when others=>next_state <= S0;
end case;
end process;

process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            qout<=(others=>'0');
        else
            if(curr_state =S5)then
                qout <=din;
            else
                qout <= "00010000";
            end if;
        end if;
    end if;
end process;
end rtl;

```

11.3.9 顺序码编码方式的 Moore 型状态机（Verilog）

```

// State Machine with sequential code encoding
module gowin_fsm (clk,rst,sel,qout);
    input clk;
    input rst;

```

```
input [1:0]sel;
output reg [7:0] qout;
localparam S0 = 3'b000;
localparam S1 = 3'b001;
localparam S2 = 3'b010;
localparam S3 = 3'b011;
localparam S4 = 3'b100;
localparam S5 = 3'b101;

reg [2:0] curr_state;
reg [2:0] next_state;
always @(posedge clk or posedge rst) begin
    if(rst)
        curr_state <= S0;
    else
        curr_state <= next_state;
end

always @(*) begin
    next_state = S0;
    case(curr_state)
        S0 :begin
            if(sel == 2'b00)
                next_state = S1;
            else
                next_state = S0;
            end

        S1 :begin
            if(sel == 2'b01)
                next_state = S2;
            else
                next_state = S1;
            end
    end
```

```
S2 :begin
    if(sel == 2'b10)
        next_state = S3;
    else
        next_state = S2;
    end

S3 :begin
    if(sel == 2'b11)
        next_state = S4;
    else
        next_state = S3;
    end

S4 :begin
    if(sel == 2'b00)
        next_state = S4;
    else
        next_state = S5;
    end

S5 :begin
    if(sel == 2'b01)
        next_state = S5;
    else
        next_state = S0;
    end

    default:next_state = S0;
endcase
end

always @(posedge clk or posedge rst) begin
    if(rst) begin
        qout <= 0;
    end
    else begin
        if(curr_state==S5)begin
            qout <= 8'h10;
        end
    end
end
```

```

        end
    end
end

```

```

endmodule

```

11.3.10 顺序码编码方式的 Moore 型状态机（VHDL）

```

-- State Machine with sequential code encoding

```

```

library ieee;

```

```

use ieee.std_logic_1164.all;

```

```

use ieee.std_logic_unsigned.all;

```

```

entity gowin_fsm is

```

```

    port(

```

```

        clk : in std_logic;

```

```

        rst : in std_logic;

```

```

        sel : in std_logic_vector(1 downto 0);

```

```

        qout: out std_logic_vector(7 downto 0)

```

```

    );

```

```

end entity;

```

```

architecture rtl of gowin_fsm is

```

```

    signal curr_state:std_logic_vector(2 downto 0);

```

```

    signal next_state:std_logic_vector(2 downto 0);

```

```

    constant S0 : std_logic_vector := "000";

```

```

    constant S1 : std_logic_vector := "001";

```

```

    constant S2 : std_logic_vector := "010";

```

```

    constant S3 : std_logic_vector := "011";

```

```

    constant S4 : std_logic_vector := "100";

```

```

    constant S5 : std_logic_vector := "101";

```

```

begin

```

```

    process(clk,rst)begin

```

```

        if(rst='1')then

```

```

            curr_state<=S0;

```

```

        else

```

```
        if(rising_edge(clk))then
            curr_state<=next_state;
        end if;
    end if;
end process;
```

```
process(sel,curr_state)begin
    next_state <= S0;
    case curr_state is
        when S0 =>
            if(sel ="00")then
                next_state <= S1;
            else
                next_state <= S0;
            end if;
        when S1 =>
            if(sel ="01")then
                next_state <= S2;
            else
                next_state <= S1;
            end if;
        when S2 =>
            if(sel ="10")then
                next_state <= S3;
            else
                next_state <= S2;
            end if;
        when S3 =>
            if(sel ="11")then
                next_state <= S4;
            else
                next_state <= S3;
            end if;
        when S4 =>
```

```

        if(sel ="00")then
            next_state <= S4;
        else
            next_state <= S5;
        end if;
    when S5 =>
        if(sel ="01")then
            next_state <= S5;
        else
            next_state <= S0;
        end if;
    when others=>next_state <= S0;
end case;
end process;

process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            qout<=(others=>'0');
        else
            if(curr_state =S5)then
                qout <= "00010000";
            end if;
        end if;
    end if;
end process;
end rtl;

```

11.3.11 顺序码编码方式的 Mealy 型状态机（Verilog）

```

// State Machine with sequential code encoding
module gowin_fsm (clk,rst,sel,din,qout);
    input clk;
    input rst;
    input [1:0]sel;
    input [7:0] din;

```

```
output reg [7:0] qout;
localparam S0 = 3'b000;
localparam S1 = 3'b001;
localparam S2 = 3'b010;
localparam S3 = 3'b011;
localparam S4 = 3'b100;
localparam S5 = 3'b101;

reg [2:0] curr_state;
reg [2:0] next_state;
always @(posedge clk or posedge rst) begin
    if(rst)
        curr_state <= S0;
    else
        curr_state <= next_state;
end

always @(*) begin
    next_state = S0;
    case(curr_state)
        S0 :begin
            if(sel == 2'b00)
                next_state = S1;
            else
                next_state = S0;
            end

        S1 :begin
            if(sel == 2'b01)
                next_state = S2;
            else
                next_state = S1;
            end

        S2 :begin
```

```
        if(sel == 2'b10)
            next_state = S3;
        else
            next_state = S2;
        end

    S3 :begin
        if(sel == 2'b11)
            next_state = S4;
        else
            next_state = S3;
        end
    S4 :begin
        if(sel == 2'b00)
            next_state = S4;
        else
            next_state = S5;
        end
    S5 :begin
        if(sel == 2'b01)
            next_state = S5;
        else
            next_state = S0;
        end
        default:next_state = S0;
    endcase
end
always @(posedge clk or posedge rst) begin
    if(rst) begin
        qout <= 0;
    end
    else begin
        if(curr_state==S5)begin
            qout <= din;
        end
    end
end
```

```

        else begin
            qout <= 8'h10;
        end
    end
end

endmodule

```

11.3.12 顺序码编码方式的 Mealy 型状态机（VHDL）

-- State Machine with sequential code encoding

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
use ieee.std_logic_unsigned.all;
```

```
entity gowin_fsm is
```

```
    port(
```

```
        clk : in std_logic;
```

```
        rst : in std_logic;
```

```
        sel : in std_logic_vector(1 downto 0);
```

```
        din : in std_logic_vector(7 downto 0);
```

```
        qout: out std_logic_vector(7 downto 0)
```

```
    );
```

```
end entity;
```

```
architecture rtl of gowin_fsm is
```

```
    signal curr_state:std_logic_vector(2 downto 0);
```

```
    signal next_state:std_logic_vector(2 downto 0);
```

```
    constant S0 : std_logic_vector := "000";
```

```
    constant S1 : std_logic_vector := "001";
```

```
    constant S2 : std_logic_vector := "010";
```

```
    constant S3 : std_logic_vector := "011";
```

```
    constant S4 : std_logic_vector := "100";
```

```
    constant S5 : std_logic_vector := "101";
```

```
begin
```

```
    process(clk,rst)begin
```

```
    if(rst='1')then
        curr_state<=S0;
    else
        if(rising_edge(clk))then
            curr_state<=next_state;
        end if;
    end if;
end process;
```

```
process(sel,curr_state)begin
    next_state <= S0;
    case curr_state is
        when S0 =>
            if(sel ="00")then
                next_state <= S1;
            else
                next_state <= S0;
            end if;
        when S1 =>
            if(sel ="01")then
                next_state <= S2;
            else
                next_state <= S1;
            end if;
        when S2 =>
            if(sel ="10")then
                next_state <= S3;
            else
                next_state <= S2;
            end if;
        when S3 =>
            if(sel ="11")then
                next_state <= S4;
            else
```

```
        next_state <= S3;
    end if;
when S4 =>
    if(sel ="00")then
        next_state <= S4;
    else
        next_state <= S5;
    end if;
when S5 =>
    if(sel ="01")then
        next_state <= S5;
    else
        next_state <= S0;
    end if;
    when others=>next_state <= S0;
end case;
end process;

process(clk)begin
    if(rising_edge(clk))then
        if(rst='1')then
            qout<=(others=>'0');
        else
            if(curr_state =S5)then
                qout <=din;
            else
                qout <= "00010000";
            end if;
        end if;
    end if;
end process;
end rtl;
```

